SQL SERVER

TIPS & TRICKS - II

By Priyanka Agarwal

second edition

Published by MeetCoogle.com

SQL Server - Tips and Tricks II

By Priyanka Agarwal

Published by MeetCoogle.com

© MeetCoogle. All Rights Reserved, except where otherwise noted.

About the Book

SQL Server - Tips and Tricks II book has solutions of some real time SQL Server problems that a developer face in most of the real time projects. All solutions are written with best practices and comes with complete to-the-point description and source code.

You know the basics of the SQL query language, yet you feel you aren't taking full advantage of SQL's expressive power. You'd like to learn how to do more work with SQL inside the database before pushing data across the network to your applications.

Let's face it, SQL is a deceptively simple language to learn, and many database developers never go far beyond the simple statement: SELECT columns FROM table WHERE conditions. But there is so much more you can do with the language. In the SQL Server - Tips and Tricks, experienced SQL developer Priyanka Agarwal shares her favorite SQL techniques and tricks to take your SQL skills to the next level. As you develop these skills, you will use either Microsoft SQL server to execute SQL statements. Everyone reading this book can jump right in with writing SQL statements in MS SQL Server with great ease.

It's easy to find basic SQL syntax and keyword information online. What's hard to find is challenging, well-designed, real-world problems—the type of problems that come up all the time when you're dealing with data. Learning how to solve these problems will give you the skill and confidence to step up in your career.

With SQL Server - Tips and Tricks, you can get that level of experience by solving sets of targeted problems. These aren't just problems designed to give an example of specific syntax. These are the most common problems you encounter when you deal with data.

You will get real world practice, with real world data. I'll teach you how to "think" in SQL, how to analyze data problems, figure out the fundamentals, and work towards a solution that you can be proud of. It contains challenging problems, which develop your ability to write high quality SQL code.

It has data analysis and reporting oriented challenges that are designed to step you through introductory, intermediate and advanced SQL Select statements, with a learn-by-doing technique.

Most textbooks and courses have some practice problems. But most often, they're used just to illustrate a particular syntax. There's no filtering on what's most useful, and what the most common issues are. What you'll get with SQL Practice Problems is the problems that illustrate some the most common challenges you'll run into with data, and the best, most useful techniques to solve them.

Find out how to migrate databases, work with constraints, create stored procedures, triggers, functions, views and cursors and configure effective queries. Security, monitoring, and tuning techniques are also covered in this volume.

Want to be an Author?

We will make you one!
(And yes, you can earn from it too!)

Share your story with us and let us help recognize your talent and promote your story in the world's biggest bookstores.



Opportunity for Debutante Author

Actual Price: ~~13000 INR or 200 USD~~
Discounted Price: 2999 INR or 50 USD

It’s Not Too Late to Become a Author

Our platform, shares everything we’ve learned about what it takes to start writing for a living — and how you can get started today.

Let the world burn through you. Throw the prism light, white hot, on paper.”

Come along --> Become a member --> Publish yourself as an Author.

There is no greater agony than bearing an untold story inside you.

Sounds Complex? Don’t Worry.

Talk to our publishing consultant

Call us at 91-9711899992

Email us at sales@meetcoogle.com

Join us at MeetCoogle.com

Also visit our FaceBook page MeetCoogle

Table of Contents

About the Book
Want to be an Author?
Sounds Complex? Don't Worry.
How to find values with Leading or Trailing Spaces in SQL Server Table
How to search for a String in all Columns in all tables in SQL Server Database
    Solution:
How to Find Percentage of Empty or Blank Values in every Column of all the Tables in SQL Server Database
    Scenario:
    Solution:
How to Find Percentage of Null Values in every Column of all the Tables in SQL Server Database
    Scenario:
    Solution:
How to Search in all Columns for all tables in a database for Date Value in SQL Server
    Scenario:
    Solution:
How to compare two tables for Column Names in SQL Server
    Scenario:
    Solution:
How to copy Some Tables or all Tables to Destination Database in SQL Server
    Scenario:
    Solution:
How to Alter Column from Null to Not Null in SQL Server Table
    Scenario:
    Solution:
Get List of All Null and Not Null Columns in SQL Server Database
    Scenario:
    Solution:

How to Alter column from Not Null to Null in SQL Server Table
Scenario:
Solution:
How to Generate Add Column Statement for all the tables in a Database in SQL Server
Scenario:
Solution:
Constraints in SQL Server
How to create Not Null Constraint on Column in SQL Server Table
Scenario:
Solution:
How to List all Missing Identity Values for all Tables in SQL Server Database
Scenario:
Solution:
How to create table in all the databases in SQL Server
How to Drop Table from All the Databases from SQL Server
How to create View in all the databases in SQL Server
How to drop View from Multiple or All Databases in SQL Server
How to Create Same Stored Procedure on All Databases in SQL Server
How to drop Stored Procedure from All or Multiple databases in SQL Server
How to check when the view definition was refreshed in SQL Server
How to generate script to REFRESH all views in a SQL Server Database
How to check if User is Enabled or Disabled in all the databases in SQL Sever
How to check if table exists in all databases in SQL Server
Scenario:
Solution:
How to get Maximum and Minimum Length for each Column Values for all the tables in SQL Server Database
How to Run Stored Procedure against Multiple Databases in SQL Server
How to use ANY / SOME Logical Operator in SQL Server
How to use ALL Logical Operator in SQL Server

How to use EXISTS Logical Operator in SQL Server
What is BETWEEN Logical Operator in SQL Server
What is LIKE Logical Operator in SQL Server
Filtering by OFFSET-FETCH Options in Select query
How to get random rows from SQL Server Table
How to use Case Statement for Conditional Formatting in Select Query
Understand Column Alias in Select Query in SQL Server
How to use Top with Ties in SQL Server
How to get Top / Bottom N Records from SQL Server Table in SQL Server
How to restore Databases from backups files automatically in SQL Server
How to quickly write Select Query in SQL Server
How to generate Insert Statements from Text Files for SQL Server Table in SQL Server

More to Come

How to find values with Leading or Trailing Spaces in SQL Server Table

Scenario:

You are working as SQL Server Developer / TSQL Developer with Mortgage Company. You are preparing script for data cleaning. You need to find all the records with leading and trailing spaces. Once you found them then you need to write update statement to remove leading and trailing spaces.

Solution:

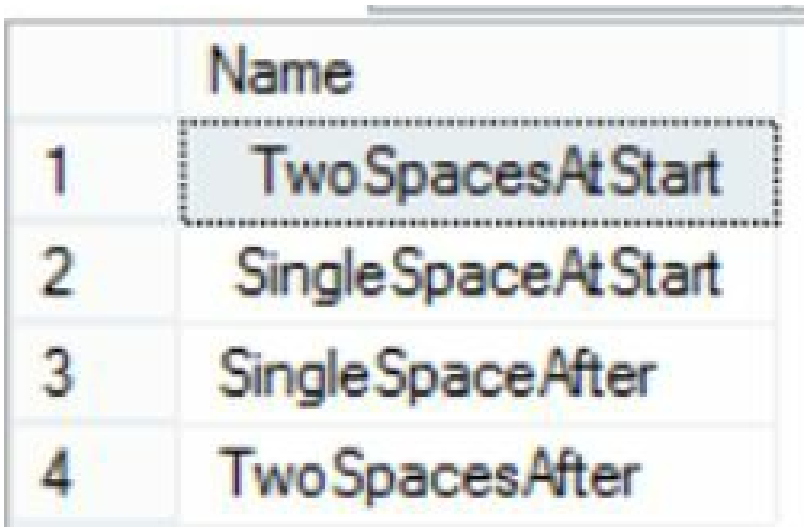
Let's create our test temp table with some test data. I have included leading and trailing space in 4 values.

```
--Create Test Table

Create table #Temp( Name VARCHAR(100))
Go
Insert into #Temp
Select 'NoSpace' AS NAme
Union
Select ' SingleSpaceAtStart' AS Name
Union
Select ' TwoSpacesAtStart' AS Name
Union
Select 'SingleSpaceAfter ' AS Name
Union
Select 'TwoSpacesAfter  ' as Name
```

Let's go ahead and run our Select Query to find Leading and Trailing spaces in our data.

```
Select * from #Temp
where name like '% ' --Will provide us values with Trailing space/s
or name like ' %'     --Will provide us values with leading Space/s
```



	Name
1	TwoSpacesAtStart
2	SingleSpaceAtStart
3	SingleSpaceAfter
4	TwoSpacesAfter

Our query returned only 4 records, as there is no leading or trailing space for first record.
Let's use the update statement to update the records and remove the leading and trailing spaces.

```
--Update the records and remove leading and trailing spaces

Update #Temp
Set Name=LTRIM(RTRIM(Name))
where name like '% '
or name like ' %'
```

Four records will be updated. If I will run select query again to check the leading or trailing spaces in column, I will get no records as the spaces are removed and records are updated.

Go ahead and run just to confirm

```
Select * from #Temp
where name like '% ' --Will provide us values with Trailing space/s
or name like ' %'    --Will provide us values with leading Space/s
```

```
Select * from #Temp
where name like '% ' --Will provide us values with Trailing space/s
or name like ' %'--Will provide us values with leading Space/s
```

100 %

Results | Messages

	Name

How to search for a String in all Columns in all tables in SQL Server Database

Scenario:

You are working as SQL Server developer / TSQL Developer for finical firm. You are working on analysis data from one of the database. You need to find String let say "Aamir" in all the columns of all the table in SQL server database.

Solution:

This analysis can really help us to find if we are strong a lot of duplicate data and what type of index will work better for us. We often get this type of request from business where they like to check the data in all the tables and further dig into it if found.

We are going to use Cursor to find all the string in all the columns in all the tables in SQL Server Database. As this query is going to run for all the columns of datatype char, nchar, varchar, nvarchar,text and ntext. This can be expensive. If your database is small, you might be good to run.

In cases where your database is big in size, you might want to take the backup and restore on server where users are not connected and run your query there. Or run during those hours when you have less or almost no activity.

```
USE YourDBName
GO

Declare @SearchString VARCHAR(100)

--Provide the String here. I am using Aamir for search
SET @SearchString='aamir'
DECLARE @DatabaseName VARCHAR(100)
DECLARE @SchemaName VARCHAR(100)
DECLARE @TableName VARCHAR(100)
DECLARE @ColumnName VARCHAR(100)
DECLARE @FullyQualifiedTableName VARCHAR(500)
Declare @DataType VARCHAR(50)

--Create Temp Table to Save Results
IF OBJECT_ID('tempdb..#Results') IS NOT NULL
   DROP TABLE #Results

CREATE TABLE #Results (
   DatabaseName VARCHAR(100)
   ,SchemaName VARCHAR(100)
```

```
    ,TableName VARCHAR(100)
    ,ColumnName VARCHAR(100)
    ,ColumnDataType VARCHAR(50)
    ,TotalTableRowCount INT
    ,StringOccuranceRecordCount INT
    )

DECLARE Cur CURSOR
FOR
SELECT C.Table_CataLog
    ,C.Table_Schema
    ,C.Table_Name
    ,C.Column_Name
    ,'[' + C.Table_CataLog + ']' + '.[' + C.Table_Schema + '].'
    + '[' + C.Table_Name + ']' AS FullQualifiedTableName,
    C.Data_Type
FROM information_schema.Columns C
INNER JOIN information_Schema.Tables T ON C.Table_Name =
T.Table_Name
    AND T.Table_Type = 'BASE TABLE'
    and (C.Data_Type like '%CHAR%'
    or C.Data_Type like '%Text%')

OPEN Cur
FETCH NEXT
FROM Cur
INTO @DatabaseName
    ,@SchemaName
    ,@TableName
    ,@ColumnName
    ,@FullyQualifiedTableName
    ,@DataType

WHILE @@FETCH_STATUS = 0
BEGIN
    DECLARE @SQL VARCHAR(MAX) = NULL
```

```
    SET @SQL = ' Select ''' + @DatabaseName + ''' AS DatabaseName, '''
    + @SchemaName + ''' AS TableName,
      ''' + @TableName + ''' AS SchemaName,
      ''' + @ColumnName + ''' AS ColumnName,
      ''' + @DataType + ''' AS ColumnName,
      (Select count(*) from ' + @FullyQualifiedTableName + ' With (Nolock))
      AS TotalTableRowCount,
      count(*) as StringOccuranceRecordCount from ' +
@FullyQualifiedTableName
      + 'With (Nolock) Where  '+@ColumnName+' like '''+'%'+
@SearchString+'%'''

    -- Print @SQL
    INSERT INTO #Results
    EXEC (@SQL)

    FETCH NEXT
    FROM Cur
    INTO @DatabaseName
        ,@SchemaName
        ,@TableName
        ,@ColumnName
        ,@FullyQualifiedTableName
        ,@DataType
END
CLOSE Cur
DEALLOCATE Cur

SELECT *,
Cast((StringOccuranceRecordCount/Cast(TotalTableRowCount as
Numeric(13,1)))*100
AS Numeric(4,1)) AS StringOccurancePercentPerColumn
from #Results
Where StringOccuranceRecordCount<>0
--Drop table #Results
```

Here are my test results

	DatabaseName	SchemaName	TableName	ColumnName	ColumnDataType	TotalTableRowCount	StringOccuranceRecordCount	StringOccurancePercentPerColumn
1	TestDB1	dbo	mytestTable	name	varchar	5	1	20.0
2	TestDB1	dbo	mytestTable1	name	varchar	2	1	50.0
3	TestDB1	dbo	TextTable	name	text	2	1	50.0

How to Find Percentage of Empty or Blank Values in every Column of all the Tables in SQL Server Database

Scenario:

You are working as SQL Server developer / TSQL Developer for Insurance Company. You need to perform some analysis of data in one of the database. You are asked to provide Percentage of Empty or Blank Values in every Column of all the tables in Database. Don't mix Empty or Blank values with Null values.

As blank or empty value ('') can be saved in string type columns with data type such as char, nchar, varchar, nvarchar,text and ntext. We will be checking only in columns which has these data types.

Solution:

This information can be really helpful for analysis and we can see if we are loading or entering correct data. If corrections are required after analysis we can write update statements to fix the data.

For this requirement we will be using Cursor. As it is going to run on each column which has data type char, nchar, varchar, nvarchar, text or ntext. It is going to be expensive. If your database is small, you might be fine to run anytime. If you have a big database and it is used by many users or processes, you might want to run this code on weekends or after hours.

One other way can be, take the full back of your production database, restore to another server maybe UAT and run the script there.

```
USE YourDatabaseName
GO

DECLARE @DatabaseName VARCHAR(100)
DECLARE @SchemaName VARCHAR(100)
DECLARE @TableName VARCHAR(100)
DECLARE @ColumnName VARCHAR(100)
DECLARE @FullyQualifiedTableName VARCHAR(500)
Declare @DataType VARCHAR(50)

--Create Temp Table to Save Results
IF OBJECT_ID('tempdb..#Results') IS NOT NULL
   DROP TABLE #Results

CREATE TABLE #Results (
   DatabaseName VARCHAR(100)
   ,SchemaName VARCHAR(100)
   ,TableName VARCHAR(100)
   ,ColumnName VARCHAR(100)
   ,ColumnDataType VARCHAR(50)
   ,TotalTableRowCount INT
   ,EmptyRecordCount INT
```

```
    )

DECLARE Cur CURSOR
FOR
SELECT C.Table_CataLog
    ,C.Table_Schema
    ,C.Table_Name
    ,C.Column_Name
    ,'[' + C.Table_CataLog + ']' + '.[' + C.Table_Schema + '].'
    + '[' + C.Table_Name + ']' AS FullQualifiedTableName,
    C.Data_Type
FROM information_schema.Columns C
INNER JOIN information_Schema.Tables T ON C.Table_Name =
T.Table_Name
    AND T.Table_Type = 'BASE TABLE'
    and (C.Data_Type like '%CHAR%'
    or C.Data_Type like '%Text%')


OPEN Cur
FETCH NEXT
FROM Cur
INTO @DatabaseName
    ,@SchemaName
    ,@TableName
    ,@ColumnName
    ,@FullyQualifiedTableName
    ,@DataType

WHILE @@FETCH_STATUS = 0
BEGIN
    DECLARE @SQL VARCHAR(MAX) = NULL
    DECLARE @ColumnNameTRIM VARCHAR(50)
    IF @DataType like '%Text%'
    BEGIN
    SET @ColumnNameTRIM='LTRIM(RTRIM(CAST('+@ColumnName + '
AS NVARCHAR(MAX))))'
```

```
    END
    ELSE
    BEGIN
    SET @ColumnNameTRIM='LTRIM(RTRIM('+@ColumnName + '))'
    END

    SET @SQL = ' Select ''' + @DatabaseName + ''' AS DatabaseName, '''
    + @SchemaName + ''' AS TableName,
      ''' + @TableName + ''' AS SchemaName,
      ''' + @ColumnName + ''' AS ColumnName,
      ''' + @DataType + ''' AS ColumnName,
      (Select count(*) from ' + @FullyQualifiedTableName + ' with (nolock))
      AS TotalTableRowCount,
      count(*) as EmptyRecordCount from ' + @FullyQualifiedTableName
      + 'with (nolock) Where  '+@ColumnNameTRIM+'=''''

    Print @SQL
    INSERT INTO #Results
    EXEC (@SQL)

    FETCH NEXT
    FROM Cur
    INTO @DatabaseName
        ,@SchemaName
        ,@TableName
        ,@ColumnName
        ,@FullyQualifiedTableName
        ,@DataType
END
CLOSE Cur
DEALLOCATE Cur

SELECT *,
Cast((EmptyRecordCount/Cast(TotalTableRowCount as Numeric(13,1)))*100
AS Numeric(4,1))
AS EmptyValuesPercentPerColumn
from #Results
```

```
Where EmptyRecordCount<>0

--drop table #Results
```

Here are the result of above query for one of my database.

	DatabaseName	SchemaName	TableName	ColumnName	ColumnDataType	TotalTableRowCount	EmptyRecordCount	EmptyValuesPercentPerColumn
1	TestDB1	dbo	Customer	name	varchar	5	1	20.0
2	TestDB1	dbo	Table1	fname	varchar	3	2	66.7
3	TestDB1	dbo	Table1	lname	varchar	3	1	33.3
4	TestDB1	dbo	mytestTable1	name	varchar	2	1	50.0
5	TestDB1	dbo	TextTable	name	text	2	1	50.0

How to Find Percentage of Null Values in every Column of all the Tables in SQL Server Database

Scenario:

You are working as SQL Server / TSQL Developer in Auto Insurance Company. You are working on data validation and data analysis for your project. You got this requirement in which you need to find the Percentage of Null values in every column for all the tables in SQL Server Database.

This can be really helpful in analysis so you would know how much data you are really getting or you are getting no values (Null) and plan the storage. Also depending upon the analysis you might find some records those should not be Null but you have receive Null for those Columns, so data corrections can be next step.

Solution:

We are going to use Cursor to find percentage and Null Row Count in each of the column in all the tables in SQL Server Database. This process can take long time to calculate as it will be running for each column in database if Column allows Nulls. If Column is Not Nullable, we don't need to check that columns as nobody can insert null values in those columns.

I will suggest to run the script on small database and take the backup of production database and restore to UAT and run query on database in UAT. In case your database is small, you might be fine. Again you don't want to hurt performance of running processes, you might want to schedule to run after hours.

```
USE YourDBName
GO

DECLARE @DatabaseName VARCHAR(100)
DECLARE @SchemaName VARCHAR(100)
DECLARE @TableName VARCHAR(100)
DECLARE @ColumnName VARCHAR(100)
DECLARE @FullyQualifiedTableName VARCHAR(500)

--Create Temp Table to Save Results
IF OBJECT_ID('tempdb..#Results') IS NOT NULL
   DROP TABLE #Results

CREATE TABLE #Results (
   DatabaseName VARCHAR(100)
   ,SchemaName VARCHAR(100)
   ,TableName VARCHAR(100)
   ,ColumnName VARCHAR(100)
   ,TotalTableRowCount INT
   ,NullRecordCount INT
   )

DECLARE Cur CURSOR
```

```
FOR
SELECT C.Table_CataLog
    ,C.Table_Schema
    ,C.Table_Name
    ,C.Column_Name
    ,'[' + C.Table_CataLog + ']' + '.[' + C.Table_Schema + '].'
    + '[' + C.Table_Name + ']' AS FullQualifiedTableName
FROM information_schema.Columns C
INNER JOIN information_Schema.Tables T ON C.Table_Name =
T.Table_Name
    AND T.Table_Type = 'BASE TABLE'
    and C.IS_Nullable='YES'


OPEN Cur

FETCH NEXT
FROM Cur
INTO @DatabaseName
    ,@SchemaName
    ,@TableName
    ,@ColumnName
    ,@FullyQualifiedTableName

WHILE @@FETCH_STATUS = 0
BEGIN
    DECLARE @SQL VARCHAR(MAX) = NULL

    SET @SQL = ' Select ''' + @DatabaseName + ''' AS DatabaseName, '''
    + @SchemaName + ''' AS TableName,
      ''' + @TableName + ''' AS SchemaName,
      ''' + @ColumnName + ''' AS ColumnName,
      (Select count(*) from ' + @FullyQualifiedTableName + ' with (nolock))
      AS TotalTableRowCount,
      count(*) as NullRecordCount from ' + @FullyQualifiedTableName
      + 'with (nolock) Where ' + @ColumnName + ' IS NULL'
```

```
    --Print @SQL
    INSERT INTO #Results
    EXEC (@SQL)

    FETCH NEXT
    FROM Cur
    INTO @DatabaseName
        ,@SchemaName
        ,@TableName
        ,@ColumnName
        ,@FullyQualifiedTableName
END

CLOSE Cur

DEALLOCATE Cur


SELECT *,
Cast((NullRecordCount/Cast(TotalTableRowCount as Numeric(13,1)))*100
AS Numeric(4,1))
AS NullPercentColumnValues
FROM #Results
where NullRecordCount <>0
--drop table #Results
```

Here are my results when I ran above query on one of my database, it will return you DatabaseName, SchemaName, TableName, ColumnName, TotalTableRowCount, NullRecordCount and Null Percentage Column Values as shown below.

DatabaseName	SchemaName	TableName	ColumnName	TotalTableRowCount	NullRecordCount	NullPercentColumnValues
TestDB1	dbo	TotalSale	SalePersonFName	44	34	77.3
TestDB1	dbo	TotalSale	SalePersonLName	44	4	9.1
TestDB1	dbo	TotalSale	ProductName	44	24	54.5
TestDB1	dbo	TotalSale	ItemsSold	44	34	77.3
TestDB1	dbo	TotalSale	SoldPrice	44	24	54.5
TestDB1	dbo	TotalSale	SoldDate	44	34	77.3
TestDB1	dbo	TotalSale	City	44	24	54.5
TestDB1	dbo	TotalSale	State	44	24	54.5
TestDB1	dbo	TotalSale	Country	44	44	100.0
TestDB1	dbo	TotalSale	Region	44	4	9.1
TestDB1	dbo	Customer_TechBrothersIT1	F4	2	2	100.0

How to Search in all Columns for all tables in a database for Date Value in SQL Server

Scenario:

You are working as SQL Server / TSQL developer and you need to write some scripts for data validation. You got this requirement in which you need to find all columns in all tables in a database for if they qualify with date value you have provided in your criteria.

We will be only searching in columns which are date, datetime or datetime2 data type
We need to get total Record Count in a table and also Row Count which meets our criteria

e.g. I want to find all the tables with column names if they have data where date columns value is ='1980-01-01'

Solution:

We can use cursor to find all the tables and columns (Date, datetime, datetime2 type) if our date value exists in those columns.
We will be looping through all the tables with column of date, datetime and datetime2 type. This can be expensive. If your database is small and have few tables, you might want to run it right away. If you have thousands of tables with a lot of data, you might want to run on weekend or when have very less user activity.

```
USE YourDBName
GO

--Provide the Date for Search Condition to @DateCondition Variable
DECLARE @DateCondition VARCHAR(50)

--Provide the data criteria here
--Sample, you can use = such as =1980-01-01
--or < or <=  such as '<=1980-01-01'
--or > or >=  such as '<=1980-01-01'

SET @DateCondition = '1980-01-01'
DECLARE @DatabaseName VARCHAR(100)
DECLARE @SchemaName VARCHAR(100)
DECLARE @TableName VARCHAR(100)
DECLARE @ColumnName VARCHAR(100)
DECLARE @FullyQualifiedTableName VARCHAR(500)
DECLARE @DateConditionFormatted VARCHAR(100)

SELECT @DateConditionFormatted = CASE
        WHEN Substring(@DateCondition, 1, 2) = '<='
           THEN Replace(@DateCondition, '<=', '<=''') + ''''
        WHEN Substring(@DateCondition, 1, 2) = '>='
           THEN Replace(@DateCondition, '>=', '>=''') + ''''
        WHEN Substring(@DateCondition, 1, 1) = '>'
           AND Substring(@DateCondition, 1, 2) != '='
           THEN Replace(@DateCondition, '>', '>''') + ''''
```

```
        WHEN Substring(@DateCondition, 1, 1) = '<'
            AND Substring(@DateCondition, 1, 2) != '='
            THEN Replace(@DateCondition, '<', '<''') + ''''
        WHEN Substring(@DateCondition, 1, 1) = '='
            THEN Replace(@DateCondition, '=', '=''') + ''''
        END

--Print @DateConditionFormatted
--Create Temp Table to Save Results

IF OBJECT_ID('tempdb..#Results') IS NOT NULL
    DROP TABLE #Results

CREATE TABLE #Results (
    DatabaseName VARCHAR(100)
    ,SchemaName VARCHAR(100)
    ,TableName VARCHAR(100)
    ,ColumnName VARCHAR(100)
    ,SearchedCondition VARCHAR(50)
    ,TotalTableRowCount INT
    ,FoundRowCount INT
    )

DECLARE Cur CURSOR
FOR
SELECT C.Table_CataLog
    ,C.Table_Schema
    ,C.Table_Name
    ,C.Column_Name
    ,'[' + C.Table_CataLog + ']' + '.[' + C.Table_Schema + '].' + '[' +
C.Table_Name + ']' AS FullQualifiedTableName
FROM information_schema.Columns C
INNER JOIN information_Schema.Tables T ON C.Table_Name =
T.Table_Name
    AND T.Table_Type = 'BASE TABLE'
    AND C.Data_Type IN (
        'date'
```

```
        ,'datetime'
        ,'datetime2'
        )

OPEN Cur

FETCH NEXT
FROM Cur
INTO @DatabaseName
    ,@SchemaName
    ,@TableName
    ,@ColumnName
    ,@FullyQualifiedTableName

WHILE @@FETCH_STATUS = 0
BEGIN
    DECLARE @SQL VARCHAR(MAX) = NULL

    SET @SQL = ' Select ''' + @DatabaseName + ''' AS DatabaseName, ''' +
@SchemaName + ''' AS TableName,
      ''' + @TableName + ''' AS SchemaName,
      ''' + @ColumnName + ''' AS ColumnName,
      ''' + @DateCondition + ''',(Select count(*) from ' +
@FullyQualifiedTableName + '

with (nolock))
      AS TotalTableRowCount,
      count(*) as SearchRowCount from ' + @FullyQualifiedTableName + ' with
(nolock)

Where ' + @ColumnName + ' ' + @DateConditionFormatted


    --Print @SQL
    INSERT INTO #Results
    EXEC (@SQL)

    FETCH NEXT
```

```
    FROM Cur
    INTO @DatabaseName
        ,@SchemaName
        ,@TableName
        ,@ColumnName
        ,@FullyQualifiedTableName
END

CLOSE Cur

DEALLOCATE Cur

SELECT *
FROM #Results
WHERE FoundRowCount <> 0
    --drop table #Results
```

I ran above code my database and here are my results.

	DatabaseName	SchemaName	TableName	ColumnName	SearchedCondition	TotalTableRowCount	FoundRowCount
1	TestDB1	dbo	Customer	dob	=1980-01-01	4	2
2	TestDB1	dbo	mytestTable	dob	=1980-01-01	5	1

How to compare two tables for Column Names in SQL Server

Scenario:

We as developer often need to find if two tables has the same number of columns and the columns matched. We like to see information

List of Columns match in both table
Column/s which does not exists in Table1 or Source Table
Column/s which does not exists in Table2 or Destination Table

Solution:

We can use below query. I have created couple of sample tables so you can see what our query will return.

```
CREATE TABLE dbo.Table1 (
   id INT
   ,fname VARCHAR(100)
   ,lname VARCHAR(100)
   ,address VARCHAR(50)
   )
GO

CREATE TABLE dbo.Table2 (
   id INT
   ,NAME VARCHAR(100)
   ,address VARCHAR(50)
   )
```

Let's run below query after providing the Schema Names and Table Names.

```
--Provide the Schema and Table Names for comparison
DECLARE @Table1Schema VARCHAR(50) = 'dbo'
DECLARE @Table2Schema VARCHAR(50) = 'dbo'
DECLARE @Table1Name VARCHAR(50) = 'Table1'
DECLARE @Table2Name VARCHAR(50) = 'table2'
;

WITH CTE1
AS (
   SELECT *
   FROM information_schema.columns
   WHERE table_schema = @Table1Schema
      AND table_name = @Table1Name
   )
   ,CTE2
AS (
```

```
    SELECT *
    FROM information_schema.columns
    WHERE table_schema = @Table2Schema
        AND table_name = @Table2Name
    )
SELECT
    --cte1.Table_Schema,cte1.Table_Name,cte1.Column_Name,
    --cte2.Table_Schema,cte2.Column_Name,cte2.Table_Name,
    IsNull(cte1.Column_Name, cte2.Column_Name) AS ColumnName
    ,CASE
        WHEN cte1.Column_Name = cte2.Column_Name
            THEN 'Exists in Both Tables ( ' + @Table1Name + ' , ' +
@Table2Name + ' )'
        WHEN cte1.Column_Name IS NULL
            THEN 'Does not Exists in ' + @Table1Name
        WHEN cte2.Column_Name IS NULL
            THEN 'Does not Exists in ' + @Table2Name
        END AS IsMatched
FROM CTE1
FULL JOIN cte2 ON cte1.Column_Name = cte2.Column_Name
```

Output for comparison of two tables columns

	ColumnName	IsMatched
1	id	Exists in Both Tables (Table1 , table2)
2	name	Does not Exists in Table1
3	address	Exists in Both Tables (Table1 , table2)
4	lname	Does not Exists in table2
5	fname	Does not Exists in table2

How to copy Some Tables or all Tables to Destination Database in SQL Server

Scenario:

I got an email from one of the viewer who would like to copy some tables from a source database to destination database. If new tables are added, the process should copy those tables as well. As per his requirement if the tables already exists in destination database, he need to drop it and create new one as definition in source database for tables can change.

Solution:

There can be multiple solutions for this problem, today we are going to solve this by using TSQL Cursor. In our Scenario we assumed that the both database are on same servers. In your situation you might have the Databases on different servers. You will be creating linked server.

The only thing you will change in the below script add linked server before @DestinationDB . You can hard code or create a variable and add in the script.

The below code will drop existing tables in destination and then create new table and copy the data from source database table. Make sure you test the code in development before run in QA, UAT and Production.

```
USE TestDB1 --Your SourceDB
GO

DECLARE @DestinationDB VARCHAR(100)

SET @DestinationDB = 'TechIT'

DECLARE @SchemaName VARCHAR(100)
DECLARE @TableName VARCHAR(100)

DECLARE Cur CURSOR
FOR

--Choose the list of tables you would like to copy to
--New Destination Database
SELECT DISTINCT Table_Schema AS SchemaName
    ,Table_Name AS TableName
FROM information_Schema.Tables
WHERE Table_Type = 'BASE TABLE'

OPEN Cur

FETCH NEXT
```

```
FROM Cur
INTO @SchemaName
   ,@TableName

WHILE @@FETCH_STATUS = 0
BEGIN
   DECLARE @DropTableSQL NVARCHAR(MAX)

--Drop the Table in Destination if exists
   SET @DropTableSQL = '
     if exists (select * from ' + @DestinationDB +
'.INFORMATION_SCHEMA.TABLES where
      TABLE_NAME = ''' + @TableName + ''' AND TABLE_SCHEMA = ''' +
@SchemaName + ''')
   drop table [' + @DestinationDB + '].[' + @SchemaName + '].[' +
@TableName + ']'

   PRINT (@DropTableSQL)

   EXEC (@DropTableSQL)

   DECLARE @InsertSQL NVARCHAR(MAX)
--Create new table and move data to it from Source Table
   SET @InsertSQL = 'SELECT * INTO ' + '[' + @DestinationDB + '].[' +
@SchemaName + '].[' + @TableName + ']
   FROM [' + @SchemaName + '].[' + @TableName + ']'

   PRINT @InsertSQL

   EXEC (@InsertSQL)

   FETCH NEXT
   FROM Cur
   INTO @SchemaName
      ,@TableName
END

CLOSE Cur
```

```
DEALLOCATE Cur
```

Things we learnt from this code

- How to get list of tables with Schema from a database
- How to use Cursor to loop through records in TSQL
- How to use Dynamic SQL to prepare our query for each row and execute
- How to copy tables from one database to another database on same server or on linked server

How to Alter Column from Null to Not Null in SQL Server Table

Scenario:

You are working as SQL Server developer with a Bank. They have Dbo.Customer table in database with below definition.

```
Create Table dbo.Customer
(
FirstName VARCHAR(50) Not Null,
LastName VARCHAR(50),
Age SmallInt,
PhoneNumber CHAR(9),
DOB Date,
Gender CHAR(1)
)
```

As you noticed that LastName can be Null. That is the problem. Company noticed that later and always want to have the value for LastName. You are asked to do analysis and write Alter script to Change Column from Null to Not Null. What things you will consider or suggestions you will provide?

Solution:

The very first thing to consider in this scenario is to find out if any values are already inserted in the table for Last Name as Null. If that is the case, you can't really go ahead and Alter column from Null to Not Null. If you try, you will get below error.

```
Msg 515, Level 16, State 2, Line 14
Cannot insert the value NULL into column 'LastName', table
'TechIT.dbo.Customer';
column does not allow nulls. UPDATE fails.
```

It means we have to deal with Null values in this column first. You can talk with business and ask them what they would like to do for all the customer where Last Name is Null. You can find the list of customers where Last Name is Null by using below query.

```
Select * From dbo.Customer where LastName Is null
```

Business can provide different suggestions such as

- Let's update the LastName to Unknow where it is Null
- Or Update the Last Name to blank('') if it is Null
- Or Update the Last Name column to LNNP (Last Name Not Provided)
- Or they go back to customers and get their last Name to update

Depending upon the suggestions, go ahead and update the values in Last Name column. Let's say we decided to update to blank '', we can use below query to update

```
update dbo.Customer
set LastName=''
where LastName is Null
```

Now you are all set to change column in table **from Null to Not Null.**

```
Alter Table dbo.Customer
```

```
Alter Column LastName VARCHAR(50) Not Null
```

Get List of All Null and Not Null Columns in SQL Server Database

Scenario:

You are working as SQL Server Developer, You are doing some analysis and want to get the list of all columns in SQL Server Database which are Nullable or Not Nullable. How would you get this information?

Solution:

We can use system views in SQL Server to get this information. Below Query will return us Database Name, Schema Name, Table Name, Column Name, Data Type and IS_Nullable.

```
SELECT Table_CataLog AS DBName
    ,table_Name
    ,Column_Name
    ,Data_Type
    ,IS_NULLABLE
FROM INFORMATION_SCHEMA.COLUMNS

  --Uncomment the Where Clause if you want to filter the records
  --where TABLE_SCHEMA = 'dbo' and TABLE_NAME = 'TableName' and
COLUMN_NAME = 'ColumnName'
```

How to Alter column from Not Null to Null in SQL Server Table

Scenario:

You are working as SQL Server developer for Auto Insurance Company, You were asked to create a table with below definition where Middle Name has to be provided always. So you have created the Column with Not Null property. After some time, your company encountered some data, where customer had no Middle Name. They would like you to change the Column to accept Null if not provided. What scripts you will provide?

```
Create Table Customer
(FirstName VARCHAR(50) Not Null,
MiddleName VARCHAR(50) Not Null,
LastName VARCHAR(50),
Age SmallInt,
PhoneNumber CHAR(9),
DOB Date,
Gender CHAR(1)
)
```

Solution:

You can simply Alter the column by using below statement.

```
Alter table dbo.TableName
Alter Column ColumnName DataType Size Null
```

To Alter Middle Name to accept Null, we can use below statement.

```
Alter table dbo.Customer
Alter Column MiddleName VARCHAR(50) Null
```

How to Generate Add Column Statement for all the tables in a Database in SQL Server

Scenario:

You are working in Auto Insurance Company as SQL Server Developer. Your company has a database in which they have more than 300 Tables. In initial design, they forgot to add Audit columns such as CreatedBy and CreatedOn. They want you to generate Add Column CreatedBy and CreatedOn for all the Tables in a Database, How would you do that?

Solution:

There are many ways to generate the SQL Statements for above requirement, we are going to keep it simple and quick. We will use Select Query to generate Add Column Statements for all the tables.

Let's generate the scripts in a way so we have to make very small changes every time and we can use anytime to add new column. I have declared two variables, @ColumnName and @ColumnDataType. Once we will provide the values for variables. The Select query is going to check all the tables and if column does not exist for the table, it will generate Add Column statement.

```
--Declare Variables for Column Name and Data Type

Declare @ColumnName VARCHAR(100)
Declare @ColumnDataType VARCHAR(50)

--Set the Values for Variables

SET @ColumnName='CreatedBy'
SET @ColumnDataType='VARCHAR(50)'

--Run the Query and copy results and paste in new window to run.

Select 'ALTER Table ['+Table_Schema+'].['+Table_Name+'] '
+'Add '+@ColumnName+' '+@ColumnDataType AS AddColumnQuery
from Information_Schema.Tables T
where T.Table_Type='BASE TABLE'
And Not exists (
Select 1 from INFORMATION_SCHEMA.COLUMNS C
WHERE C.TABLE_Name=T.Table_Name and
Column_Name=@ColumnName)
```

Run above query and copy the results to new Query Windows, Select the Database on which you would like to run and execute.

AddColumnQuery
ALTER Table [dbo].[Cus] Add CreatedBy VARCHAR(50)
ALTER Table [dbo].[Customer] Add CreatedBy VARCHAR(50)

Constraints in SQL Server

In simple words, Constraints define rules those we implement to avoid insertion of any data that does not follow the rule.
By using the Constraint, we can validate the data and if complies the rule, It will be inserted in table otherwise rolled back.
Constraints are used to maintain Data Integrity. They make sure the data entered follow the rules we have defined.

There are different types of Constraints available in SQL Server.

1. Not Null Constraint
2. Check Constraint
3. Default Constraint
4. Unique Constraints
5. Primary Key Constraint
6. Foreign Key Constraint

How to create Not Null Constraint on Column in SQL Server Table

Scenario:

You are creating customer table for one of your requirement, you want to make sure the First name Column always have the value. How will you make sure First Name Column always have the value?

Solution:

Constraints defined rules which make sure the data we insert in column follow the rules.

For above requirement we can create Not Null constraint on a column, Not Null Constraint make sure the value inserted in that column is not null. If user will try to insert row without providing the value for the column on which we have created the Not Null Constraint, The insert will fail as column cannot accept Null values.

To create Not Null Constraint on Column in a Table, we have to Add Not Null after the Data Type as shown below for First Name Column.

```
Create Table Customer
(FirstName VARCHAR(50) Not Null,
LastName VARCHAR(50),
Age SmallInt,
PhoneNumber CHAR(9),
DOB Date,
Gender CHAR(1)
)
```

You can see that I did not provide Not Null Constraint for other columns, that means those column can accept Null values or Unknown value.

Let's insert below rows and see what happen

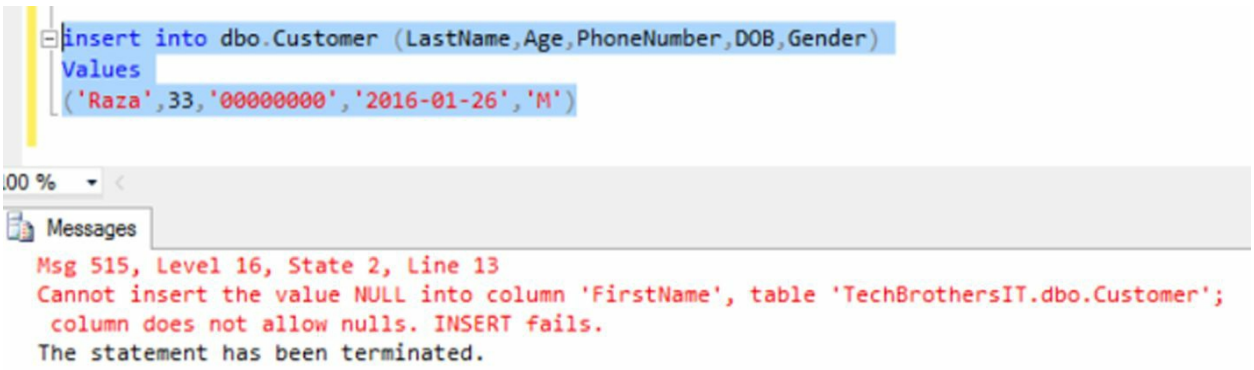
```
insert into dbo.Customer (
FirstName,LastName,Age,PhoneNumber,DOB,Gender)
Values
('Aamir','Shahzad',Null,Null,Null,Null)
,(Null,'Raza',33,'00000000',Null,'M')
```

As you can see that the transaction failed as we can't insert the Null in First Name column, No Record will be inserted.



Let's consider second Insert, Where we are not inserting anything in First Name column. Still the insert will fail, as when you don't provide the column in insert list, it try to insert Null value for that column and we have created Not Null Constraint on First Name column.

```
insert into dbo.Customer (LastName,Age,PhoneNumber,DOB,Gender)
Values
('Raza',33,'00000000','2016-01-26','M')
```



How to List all Missing Identity Values for all Tables in SQL Server Database

Scenario:

You are working as SQL Server Developer / TSQL developer for Law firm. You are doing working on data validation / data analysis project. Once of the task you got for analysis, you need to list all the missing Identity values for all the tables in SQL Server Database.

Solution:

We will use cursor to loop through all the tables in a database which has identity column. We will load the missing identity values for each table in temp table and print at the end. We will get Database Name, Schema Name, Table Name, Identity Column and Missing Identity Values.

If you would like to run the script for single table. You can edit Where clause and include Table Name.

```
USE YourDBName

go

DECLARE @SchemaName VARCHAR(100)
DECLARE @TableName VARCHAR(100)
DECLARE @DatabaseName VARCHAR(100)
DECLARE @IdentyColumnName VARCHAR(100)

--Create Temp Table to Save Results

IF Object_id('tempdb..##Results') IS NOT NULL
  DROP TABLE ##results

CREATE TABLE ##results
  (
     schemaname          VARCHAR(100),
     tablename           VARCHAR(100),
     identitycolumname   VARCHAR(100),
     identitymissingvalue INT
  )

DECLARE cur CURSOR FOR
  SELECT Schema_name(schema_id)   AS SchemaName,
         Object_name(c.object_id) AS TableName,
         C.NAME                   AS IdentityColumnName
  FROM   sys.columns c
         INNER JOIN sys.tables t
```

```
            ON c.object_id = t.object_id
  WHERE  is_identity = 1

OPEN cur

FETCH next FROM cur INTO @SchemaName, @TableName, @IdentyColumn

WHILE @@FETCH_STATUS = 0
  BEGIN
     DECLARE @SQL VARCHAR(max)=NULL
     DECLARE @InitialIDValue INT=1
     DECLARE @MaxIdentityId INT
     DECLARE @MAXID TABLE
      (
        maxidentityid INT
      )

     DELETE FROM @MAXID

     DECLARE @SQLIdentity NVARCHAR(max)

     SET @SQLIdentity='SELECT MAX(' + @IdentyColumnName + ') From ['
                 + @SchemaName + '].[' + @TableName + ']'

     INSERT INTO @MAXID
     EXEC(@SQLIdentity)

     --Select * from @MAXID
     SET @MaxIdentityId=(SELECT *
                   FROM   @MAXID)

     PRINT @MaxIdentityId

     WHILE @InitialIDValue <= @MaxIdentityId
      BEGIN
         DECLARE @SQLQuery NVARCHAR(max)=NULL
         DECLARE @InitialIDValueCHAR VARCHAR(10)=NULL
```

```
        SET @InitialIDValueCHAR=Cast(@InitialIDValue AS VARCHAR(10))

        PRINT @InitialIDValueCHAR

        SET @SQLQuery=' IF NOT EXISTS (SELECT 1 FROM ['
                    + @SchemaName + '].[' + @TableName
                    + ']                    WHERE ['
                    + @IdentyColumnName + '] = '
                    + @InitialIDValueCHAR + ')
 INSERT INTO ##results(SchemaName,TableName,IdentityColumName,Identi
 VALUES ( '''
                    + @SchemaName + '''' + ',''' + @TableName + '''' +
                    ','''
                    + @IdentyColumnName + '''' + ','''
                    + @InitialIDValueCHAR + ''')'

        PRINT @SQLQuery

        EXEC (@SQLQuery)

        SET @InitialIDValue = @InitialIDValue + 1
      END

    FETCH next FROM cur INTO @SchemaName, @TableName, @IdentyCol
 END

CLOSE cur

DEALLOCATE cur

SELECT Db_name() AS DatabaseName,
      schemaname,
      tablename,
      identitycolumname,
      identitymissingvalue
FROM   ##results
ORDER  BY tablename
--drop table #Results
```

I ran above query on my Database to Identity missing identity values for all the tables and got below results.

	DatabaseName	SchemaName	TableName	IdentityColumname	IdentityMissingValue
1	TestDB1	dbo	Dups	id	2
2	TestDB1	dbo	Dups	id	3
3	TestDB1	Sales	MyTestTable	SaleID	4
4	TestDB1	dbo	MyTestTable2	MyId	3
5	TestDB1	dbo	MyTestTable2	MyId	7
6	TestDB1	dbo	Test4	id	4
7	TestDB1	dbo	Test4	id	5

How to create table in all the databases in SQL Server

Scenario:
You are working as SQL Server DBA or developer and you got this requirement" You have to create a table in all the databases on SQL Server Instance" If you have few databases, it might be each to change the Database scope in session and run the query but think about situation if you have 100's of databases and you have to create the table in every database.

Solution:
Below script can be used to create table in all the Databases on SQL server. You can also filter the databases on which you would like to run the script. Please the DDL statement for your table create in below script and you are good to go.

```
--Provide the DDL Statment that you would like to run for Create table in Each Databsae in SQL Server
   DECLARE @DDL VARCHAR(MAX)
   SET @DDL='Create table dbo.MyNewTableName (id int, name varchar(100))'
   DECLARE @DatabaseName AS VARCHAR(500)

--DECLARE CURSOR
   DECLARE CUR CURSOR
   FOR
   --Choose the DBs on which you would like to run the script
   SELECT NAME
   FROM sys.databases
   WHERE database_id > 4

--OPEN CURSOR
   OPEN CUR
```

```
--NEXT NEXT RECORD
    FETCH NEXT
    FROM CUR
    INTO @DatabaseName

    WHILE @@FETCH_STATUS = 0
    BEGIN
        DECLARE @DBName AS NVARCHAR(500);

        SET @DBName = QUOTENAME(N'' + @DatabaseName + '');

        --USE Dynamic SQL To Change DB name and run DDL statement to
create table
        EXEC (
              N'USE ' + @DBName + N'; EXEC('''+@DDL+''');'
              );

        FETCH NEXT
        FROM CUR
        INTO @DatabaseName
    END

    CLOSE CUR

    DEALLOCATE CUR
```

Once you run above script, dbo.MyNewTableName will be created in all the database which were returned by select * from sys.databases.

How to Drop Table from All the Databases from SQL Server

Scenario:
You are working as SQL server developer or SQL server DBA. You got this requirement " You need to create the scripts to drop the table from all the databases if exists in SQL Server". If you have small number of databases, let's say 5 or 10, It is not a big deal to go to each Database and drop the table. But think about a situation when you have 100's of databases and you have to drop the table from each database if exists.

Solution:
I have used the cursor with dynamic SQL to loop through the databases and drop the drop if exists.
You can filter the databases for which you want to run the script by add more filters to where clause in sys.database part of the code.

The below code is going to check if table exists and if it does, it is going to drop. I will suggest to run the script on **Development, SIT, QA environments first and then on Prod.**

```
--Provide the DDL Statement that you would like to run to drop the table if exists in Database

--Change the schema and table name in script as per your requirement
DECLARE @DDL VARCHAR(MAX)
SET @DDL='IF (EXISTS (SELECT *
            FROM INFORMATION_SCHEMA.TABLES
            WHERE TABLE_SCHEMA = ''dbo''
            AND TABLE_NAME = ''MyNewTableName''))
BEGIN
  Drop table dbo.MyNewTableName
END'
   DECLARE @DatabaseName AS VARCHAR(128)
```

```
--DECLARE CURSOR
   DECLARE CUR CURSOR
   FOR
   --Choose the DBs on which you would like to run the script
   SELECT NAME
   FROM sys.databases
   WHERE database_id > 4

--OPEN CURSOR
   OPEN CUR

--NEXT RECORD
   FETCH NEXT
   FROM CUR
   INTO @DatabaseName

   WHILE @@FETCH_STATUS = 0
   BEGIN
      DECLARE @DBName AS NVARCHAR(500);

      SET @DBName = QUOTENAME(N'' + @DatabaseName + '');

--USE Dynamic SQL To Change DB name and run DDL statement to drop table
      EXEC (
            N'USE ' + @DBName + N'; EXEC('''+@DDL+''');'
            );

      FETCH NEXT
      FROM CUR
      INTO @DatabaseName
   END

   CLOSE CUR

   DEALLOCATE CUR
```

How to create View in all the databases in SQL Server

Scenario:

You are working as SQL Server DBA or SQL Server developer and you need to create a view with same definition in multiple databases of all databases on SQL Server Instance. You can use below script to create view in multiple databases. I used cursor to loop through the list of databases. You can always filter (choose) the databases in select * from sys.databases query to create view in required databases.

Solution:

```
--Provide the DDL Statment that you would like to run for Create View in Each Databsae in SQL Server
    DECLARE @DDL VARCHAR(MAX)
    SET @DDL=
    'Create View dbo.MyTestView
    AS
    Select * from dbo.MyNewTableName'
    DECLARE @DatabaseName AS VARCHAR(500)

--DECLARE CURSOR
    DECLARE CUR CURSOR
    FOR
    --Choose the DBs on which you would like to run the script
    SELECT NAME
    FROM sys.databases
    WHERE database_id > 4

--OPEN CURSOR
    OPEN CUR

--NEXT NEXT RECORD
    FETCH NEXT
    FROM CUR
    INTO @DatabaseName

    WHILE @@FETCH_STATUS = 0
```

```
    BEGIN
        DECLARE @DBName AS NVARCHAR(128);

        SET @DBName = QUOTENAME(N'' + @DatabaseName + '');

        --USE Dynamic SQL To Change DB name and run DDL statement to
create View
        EXEC (
              N'USE ' + @DBName + N'; EXEC('''+@DDL+''');'
              );

        FETCH NEXT
        FROM CUR
        INTO @DatabaseName
    END

    CLOSE CUR

    DEALLOCATE CUR
```

How to drop View from Multiple or All Databases in SQL Server

Scenario:
You are working as SQL Server DBA or SQL Server developer, you need to come up with scripts to drop the view from many databases in one of your SQL Server Instance.

Solution:
Here is the script that can help to drop the view from multiple databases. I have used the cursor to loop through the databases. You can always modify your select query to choose required databases on which you would like to run the script. You have to change the schema name and view name and in drop view statement, the name of view.

```
--Provide the DDL Statment that you would like to run to drop the View if exists in Database
  --Change the schema and View name in script as per your requirement
    DECLARE @DDL VARCHAR(MAX)
    SET @DDL='IF (EXISTS (SELECT *
                FROM INFORMATION_SCHEMA.TABLES
                WHERE TABLE_SCHEMA = ""dbo"" -- change to your Schema
                AND  TABLE_NAME = ""MyTestView"" --Change to your View Name
                and Table_Type=""View""))
BEGIN
Drop view dbo.MyTestview
END'
    DECLARE @DatabaseName AS VARCHAR(500)

--DECLARE CURSOR
    DECLARE CUR CURSOR
    FOR
    --Choose the DBs on which you would like to run the script
```

```
    SELECT NAME
    FROM sys.databases
    WHERE database_id > 4

--OPEN CURSOR
    OPEN CUR

--NEXT NEXT RECORD
    FETCH NEXT
    FROM CUR
    INTO @DatabaseName

    WHILE @@FETCH_STATUS = 0
    BEGIN
        DECLARE @DBName AS NVARCHAR(500);

        SET @DBName = QUOTENAME(N'' + @DatabaseName + '');

--USE Dynamic SQL To Change DB name and run DDL statement to drop View
        EXEC (
                N'USE ' + @DBName + N'; EXEC('''+@DDL+''');'
                );

        FETCH NEXT
        FROM CUR
        INTO @DatabaseName
    END

    CLOSE CUR

    DEALLOCATE CUR
```

How to Create Same Stored Procedure on All Databases in SQL Server

Scenario:

We often face this situation where we need to create a same Stored Procedure in multiple databases. This script can be really usefully when you are working as SQL Server DBA and you have to deploy same Stored Procedure to all databases or multiple databases Or you are working as developer and your team asked you to prepare script which will create the Stored Procedure in all databases in SQL Server.

Solution:

I used cursor in below script to loop through the databases. You can always change your Where clause to get the list of databases on which you would like to create Stored Procedure.

It is always great idea to test your script in DEV, QA and UAT environment before deploying/ running on Production environment.

```
--Provide the DDL Statement that you would like to run for Create Stored Procedure in Each Database in SQL Server
-- Notice that if you have string in your SP, then you have to have 4 single quote around it.
    DECLARE @DDL VARCHAR(MAX)
    SET @DDL=
    'Create Procedure dbo.SP_Test
    AS
    BEGIN
    --My all statements in SP
    Select 1
    Print ""Test""
    END'
    DECLARE @DatabaseName AS VARCHAR(500)

--DECLARE CURSOR
    DECLARE CUR CURSOR
```

```
    FOR
    --Choose the DBs on which you would like to run the script
    SELECT NAME
    FROM sys.databases
    WHERE database_id > 4

--OPEN CURSOR
    OPEN CUR

--NEXT NEXT RECORD
    FETCH NEXT
    FROM CUR
    INTO @DatabaseName

    WHILE @@FETCH_STATUS = 0
    BEGIN
        DECLARE @DBName AS NVARCHAR(128);

        SET @DBName = QUOTENAME(N'' + @DatabaseName + '');

--USE Dynamic SQL To Change DB name and run DDL statement to create
Stored Procedure
        EXEC (
              N'USE ' + @DBName + N'; EXEC('''+@DDL+''');'
              );

        FETCH NEXT
        FROM CUR
        INTO @DatabaseName
    END

    CLOSE CUR

    DEALLOCATE CUR
```

How to drop Stored Procedure from All or Multiple databases in SQL Server

Scenario:
You are working as SQL Server DBA or SQL Server developer, you need to come up with scripts to drop the procedure from many databases in one of your SQL Server Instance.

Solution:
We are going to write the script how to drop the Stored Procedure in multiple or all SQL Server databases. You can filter the list of Databases on which you like to run the script by changing the where clause "Select name from sys.databases where *********"

I have cursor to loop through the databases. You need to provide the Schema Name and Stored Procedure that you would like to drop from multiple or all the databases in SQL Server.

```
--Provide the Stored Procedure Name and Schema Name that you would like to drop from multiple or all databases
   Declare @SP_Schema VARCHAR(128)
   DECLARE @SP_Name VARCHAR(128)

   --Provide Schema Name and Stored Procedure you would like to drop
   SET @SP_Schema='dbo'
   SET @SP_Name='SP_Test'


   DECLARE @DDL VARCHAR(MAX)
   SET @DDL='IF (EXISTS (Select * from sys.objects
where name='''+@SP_Name+'''
and schema_id=schema_id('''+@SP_Schema+''')
and type=''P''
and type_desc=''SQL_STORED_PROCEDURE''))
BEGIN
```

```
Drop Procedure '+@SP_Schema+'.'+@SP_Name+'
END'

Print @DDL
   DECLARE @DatabaseName AS VARCHAR(500)

--DECLARE CURSOR
   DECLARE CUR CURSOR
   FOR
   --Choose the DBs on which you would like to run the script
   SELECT NAME
   FROM sys.databases
   WHERE database_id > 4

--OPEN CURSOR
   OPEN CUR

--NEXT NEXT RECORD
   FETCH NEXT
   FROM CUR
   INTO @DatabaseName

   WHILE @@FETCH_STATUS = 0
   BEGIN
      DECLARE @DBName AS NVARCHAR(500);

      SET @DBName = QUOTENAME(N'' + @DatabaseName + '');

--USE Dynamic SQL To Change DB name and run DDL statement to drop
Stored Procedure
      EXEC (
            N'USE ' + @DBName + N'; EXEC('''+@DDL+''');'
            );

      FETCH NEXT
      FROM CUR
      INTO @DatabaseName
```

```
END

CLOSE CUR

DEALLOCATE CUR
```

How to check when the view definition was refreshed in SQL Server

Scenario:
We know that if the data type changes for columns for objects which are used in the view, we have to refresh the view definition by using sp_refreshview to update metadata for columns used in the view.

Sometime we need to know what was the last time the view definition was altered or refreshed.

Solution:
You can use system view to get that information.

```
Select * from sys.views
```

	name	object_id	principal_id	schema_id	parent_object_id	type	type_desc	create_date	modify_date
1	myview	1253579504	NULL	1	0	V	VIEW	2017-01-30 11:24:29.677	2017-01-30 11:25:27.010
2	vw_Test	1269579561	NULL	1	0	V	VIEW	2017-01-30 13:08:02.480	2017-01-30 13:08:02.480

Notice that if the view is created and never been altered or definition never been refreshed then created_date=modify_date.

Once you will refresh view definition or alter the view, then modify date will be the date time when the view as altered or definition was refreshed.

How to generate script to REFRESH all views in a SQL Server Database

Scenario:
Let's say you have made changes to data types of few of the tables and you had views created on those tables. You need to refresh the views definition after that.

Solution:
The below script can be used to generate script to refresh views.

If you have single view, you can use **sp_refreshview 'ViewName'** to refresh view definition.

To generate the script for all views in a database, you can run below statement.

```
SELECT 'sp_refreshview  '''
      + Schema_name(schema_id) + '.' + NAME + ''''
      + Char(13) + Char(10) + ' GO' AS RefreshViewQuery
FROM   sys.views
```

Click on icon "Results to Text" or press Ctrl+T and then copy the script to run for all views. You can always filter the list in your above select query if you would only want to refresh the views related to one schema or as per view names etc.

```
SELECT 'sp_refreshview  '''
       + Schema_name(schema_id) + '.' + NAME + ''''
       + Char(13) + Char(10) + ' GO' AS RefreshViewQuery
FROM   sys.views
```

100 %

Results

```
RefreshViewQuery
--------------------------------------------------------------------------------
sp_refreshview  'dbo.myview'
 GO
sp_refreshview  'dbo.vw_Test'
 GO
sp_refreshview  'TB.vw_Test'
 GO

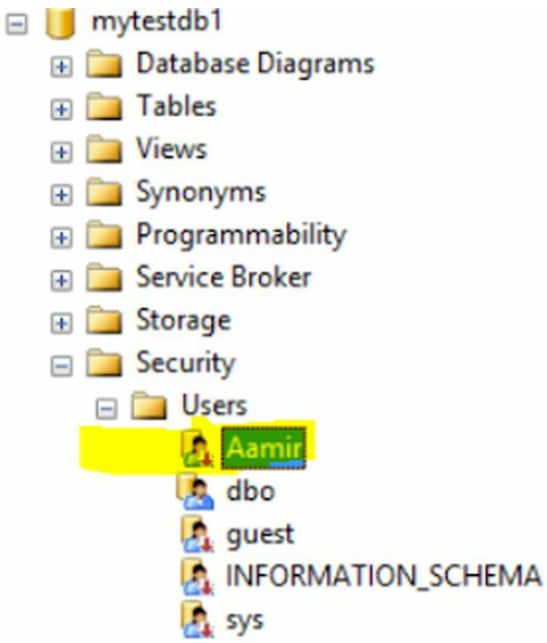
(3 row(s) affected)
```

How to check if User is Enabled or Disabled in all the databases in SQL Sever

Scenario:
You have created a login and then user in multiple databases by using that login. You need to take a look if the user is enable or disable in those databases.

Solution:
If the user is disabled in a database you will see a red arrow pointing downwards. I have disabled Aamir user as can be seen below by using

```
REVOKE CONNECT FROM Aamir
```



If you want to enable the user, you can use below statement

```
GRANT CONNECT TO UserName
```

But our goal is find out in all databases or in multiple database if the user is enabled or disabled. Also if the query does not return the Database name that

means the user does not exist in that database.

```
--Provide the user name that you would like to check if enabled or Disabled in
Databases

   DECLARE @UserName VARCHAR(128)
   SET @UserName='Aamir' -- I have provided Aamir as my User Name


 IF OBJECT_ID('tempdb..##Temp_Table') IS NOT NULL
      DROP TABLE ##Temp_Table

   CREATE TABLE ##Temp_Table (
   ServerName VARCHAR(1000),
      DatabaseName VARCHAR(500)
      ,UserName VARCHAR(500)
      ,IsEnabled INT
      )

   DECLARE @SQLStatement VARCHAR(MAX)
   SET @SQLStatement='Insert into ##Temp_Table
Select @@ServerName AS ServerName,DB_Name() AS
DatabaseName,name, hasdbaccess
FROM sys.sysusers WHERE name='''+@UserName+''''

Print @SQLStatement
   DECLARE @DatabaseName AS VARCHAR(500)

--DECLARE CURSOR
   DECLARE CUR CURSOR
   FOR
   --Choose the DBs on which you would like to run the script
   SELECT NAME
   FROM sys.databases
   WHERE database_id > 4
```

```
--OPEN CURSOR
   OPEN CUR

--NEXT NEXT RECORD
   FETCH NEXT
   FROM CUR
   INTO @DatabaseName

   WHILE @@FETCH_STATUS = 0
   BEGIN
      DECLARE @DBName AS NVARCHAR(500);

      SET @DBName = QUOTENAME(N'' + @DatabaseName + '');

--USE Dynamic SQL To Change DB name and run Check If user Enabled or
Disabled in Database
      EXEC (
            N'USE ' + @DBName + N'; EXEC('''+@SQLStatement+''');'
            );

      FETCH NEXT
      FROM CUR
      INTO @DatabaseName
   END

   CLOSE CUR

   DEALLOCATE CUR

   Select * From ##Temp_Table
```

	ServerName	DatabaseName	UserName	IsEnabled
1		ReportServer1	Aamir	1
2		ReportServerTempDB1	Aamir	1
3		YourDatabaseName	Aamir	1
4		mytestdb1	Aamir	0

How to check if table exists in all databases in SQL Server

Scenario:

You are working as SQL Server DBA / Developer. You have many databases on SQL Server Instance and you would like to check in which databases table exists or does not exists.

Solution:

```
Declare @TableName VARCHAR(128)
--Provide the Table Name that you would like to Check
SET @TableName='MyTestTable'

    --Drop Temp table if exists to save record
    IF OBJECT_ID('tempdb..##RecordCount') IS NOT NULL
        DROP TABLE ##RecordCount

    CREATE TABLE ##RecordCount (
        ServerName VARCHAR(128),
        DatabaseName VARCHAR(128)
        ,TableName VARCHAR(128)
        ,IsExists VARCHAR(10)
        )

    --Use Cursor to Loop through Databases
    DECLARE @DatabaseName AS VARCHAR(500)

    DECLARE CDCCursor CURSOR
    FOR
    SELECT NAME
    FROM sys.databases
    where database_id>4

    OPEN CDCCursor
    FETCH NEXT
    FROM CDCCursor
    INTO @DatabaseName

    WHILE @@FETCH_STATUS = 0
    BEGIN

        DECLARE @DBName AS NVARCHAR(500);

        SET @DBName = QUOTENAME(N'' + @DatabaseName + '');
```

```
        Print @DbName
        --USE Dynamic SQL To change the Database Name
        EXEC (
N'USE ' + @DBName + N'; EXEC('' 
Insert into ##RecordCount
Select @@ServerName,DB_Name() AS DatabaseName,
''''+@TableName+'''',(Select CASE WHEN count(*)>1
then ''''YES'''' ELSE ''''NO'''' END AS IsExist
from INFORMATION_SCHEMA.TABLES
where TABLE_NAME=''''+@TableName+''''
and Table_Type=''''BASE TABLE'''')
'');'
                );

        FETCH NEXT
        FROM CDCCursor
        INTO @DatabaseName
    END

    CLOSE CDCCursor

    DEALLOCATE CDCCursor

    --Return if Table Exists

    SELECT *
    FROM ##RecordCount
```

How to get Maximum and Minimum Length for each Column Values for all the tables in SQL Server Database

Scenario:
Let's say you are working as SQL Server developer / Architect and you need to design tables from existing tables. While creating existing tables nobody thought about correct data types and used NVARCHAR(MAX) or VARCHAR(MAX) , text or even ntext. Your goal is analyze the data and then come up best data types for each column. While analysis you need to know the max length for the values your columns so you can provide max data type length for new table columns.

Solution:
The below script is going to run on entire database and get the maximum and minimum length from each of the column for each of the table and return you at the end. This query can take quite a long time depending upon the number of tables, columns and records.

```
DECLARE @DatabaseName VARCHAR(100)
DECLARE @SchemaName VARCHAR(100)
DECLARE @TableName VARCHAR(100)
DECLARE @ColumnName VARCHAR(100)
DECLARE @FullyQualifiedTableName VARCHAR(500)
DECLARE @DataType VARCHAR(50)

--Create Temp Table to Save Results
IF OBJECT_ID('tempdb..#Results') IS NOT NULL
   DROP TABLE #Results

CREATE TABLE #Results (
   DatabaseName VARCHAR(100)
```

```
    ,SchemaName VARCHAR(100)
    ,TableName VARCHAR(100)
    ,ColumnName VARCHAR(100)
    ,ColumnDataType VARCHAR(50)
    ,MaxLength VARCHAR(50)
    ,MinLength VARCHAR(50)
    )

DECLARE Cur CURSOR
FOR
SELECT DB_Name() AS DatabaseName
    ,s.[name] AS SchemaName
    ,t.[name] AS TableName
    ,c.[name] AS ColumnName
    ,'[' + DB_Name() + ']' + '.[' + s.NAME + '].' + '[' + T.NAME + ']' AS
FullQualifiedTableName
    ,d.[name] AS DataType
FROM sys.schemas s
INNER JOIN sys.tables t ON s.schema_id = t.schema_id
INNER JOIN sys.columns c ON t.object_id = c.object_id
INNER JOIN sys.types d ON c.user_type_id = d.user_type_id
WHERE d.NAME LIKE '%char%'
or d.name like '%Text%'


OPEN Cur

FETCH NEXT
FROM Cur
INTO @DatabaseName
    ,@SchemaName
    ,@TableName
    ,@ColumnName
    ,@FullyQualifiedTableName
    ,@DataType

WHILE @@FETCH_STATUS = 0
```

```
BEGIN
   DECLARE @SQL VARCHAR(MAX) = NULL

   SET @SQL = ' Select ''' + @DatabaseName + ''' AS DatabaseName, ''' +
@SchemaName + ''' AS SchemaName,
      ''' + @TableName + ''' AS TableName,
      ''' + @ColumnName + ''' AS ColumnName,
      ''' + @DataType + ''' AS ColumnDataType,
      (Select MAX(LEN(CAST(' + @ColumnName + ' AS
NVARCHAR(MAX)))) from ' + @FullyQualifiedTableName + ' with
(nolock))
      AS MaxLength,
      (Select MIN(LEN(CAST(' + @ColumnName + ' AS
NVARCHAR(MAX)))) from ' + @FullyQualifiedTableName + ' with
(nolock))
      AS MinLength'

   PRINT @SQL

   INSERT INTO #Results
   EXEC (@SQL)

   FETCH NEXT
   FROM Cur
   INTO @DatabaseName
      ,@SchemaName
      ,@TableName
      ,@ColumnName
      ,@FullyQualifiedTableName
      ,@DataType
END

CLOSE Cur

DEALLOCATE Cur

SELECT *
```

```
FROM #Results
```

	DatabaseName	SchemaName	TableName	ColumnName	ColumnDataType	MaxLength	MinLength
1	Source	dbo	Customer	name	varchar	7	7
2	Source	dbo	Test	NAMEVAR	varchar	6	5
3	Source	dbo	Test	NAMENVAR	nvarchar	6	5
4	Source	dbo	testtables	column1	char	13	5
5	Source	dbo	testtables	column2	varchar	17	5
6	Source	dbo	testtables	column3	nvarchar	5	5
7	Source	dbo	testtables	column4	text	18	5
8	Source	dbo	testtables	column5	ntext	31	3

How to Run Stored Procedure against Multiple Databases in SQL Server

Scenario:
You are working as SQL Server developer. The company you are working create a new database for each of the client with same objects such as tables/Stored Procedures. You have one stored procedure with the same name let's say dbo.LoadCustomer present in all the databases. You need to execute that Stored Procedure from multiple databases. The script can be used one time or you can also run by using SQL Server agent if has to run on schedule.

Solution:
We can use Cursor to loop through list of the databases in which our Stored Procedure is present and we want to execute. Below is sample script. Please change the @SPName and filter the list of database as per your requirements.

```
--Use Cursor to Loop through Databases
DECLARE @DatabaseName AS varchar(500)
--Provide the name of SP that you want to run
DECLARE @SPName AS varchar(128) = 'dbo.loadCustomer'

DECLARE DBCursor CURSOR FOR
--Filter the list of the database in which Stored Procedure exists
SELECT
  NAME
FROM sys.databases
WHERE database_id > 4

OPEN DBCursor

FETCH NEXT
FROM DBCursor
INTO @DatabaseName

WHILE @@FETCH_STATUS = 0
```

```
BEGIN
  DECLARE @DBName AS nvarchar(500);

  SET @DBName = QUOTENAME(N'' + @DatabaseName + '');

  --USE Dynamic SQL to change the Database Name and
  --Execute Stored Procedure from that Database
  EXEC (
  N'USE ' + @DBName + N'; EXEC(''' + @SPName + '
'');'
  );

  FETCH NEXT
  FROM DBCursor
  INTO @DatabaseName
END

CLOSE DBCursor

DEALLOCATE DBCursor
```

How to use ANY / SOME Logical Operator in SQL Server

ANY Logical operator returns TRUE if any one of a set of comparisons are TRUE. ANY compares a scalar value with a single column set of values.

Note: SOME and ANY are equivalent. We are going to use ANY in our below examples.

Scenario:
Let's say that we have two tables dbo.Customer and dbo.Customer1. Both tables has the column Age. If you need to get all the records from dbo.Customer table where Age is at-least greater than one value from Age column from dbo.Customer1 table.

Solution:
We can use subquery and MIN function to write our query for above requirement. Let's create the tables first.

```
--Create Customer Table
Create table dbo.Customer
(Id int,
  FName VARCHAR(50),
  LName VARCHAR(50),
  CountryShortName CHAR(2),
  Age tinyint)
GO
--Insert Rows in dbo.Customer Table
insert into dbo.Customer
Values (
1,'Raza','M','PK',10),
(2,'Rita','John','US',12),
(3,'Sukhi','Singh',Null,25),
(4,'James','Smith','CA',60),
```

```
(5,'Robert','Ladson','US',54),
(6,'Alice','John','US',87)

--Create dbo.Customer1 table
Create table dbo.Customer1
(Id int,
  FName VARCHAR(50),
  LName VARCHAR(50),
  CountryShortName CHAR(2),
  Age tinyint)
GO
--Insert rows in dbo.Customer1 Table
insert into dbo.Customer1
Values
(7,'Raza','M','US',33),
(8,'Dita','M','US',15),
(9,'Adita','M','US',29)
```

1) Get all the records from dbo.Customer table where Age is greater than min Age value of dbo.Customer1 table by using Subquery and Min function.

```
Select * From dbo.Customer
where Age> ( Select MIN(age) from dbo.Customer1)
```

	Id	FName	LName	CountryShortName	Age
1	3	Sukhi	Singh	NULL	25
2	4	James	Smith	CA	60
3	5	Robert	Ladson	US	54
4	6	Alice	John	US	87

2) Use ANY to get required results.

We can use ANY instead of using Min function with subquery. As we want to get all rows from dbo.Customer where Age is greater than any value of Age column in dbo.Customer, We will use >Any.
>ANY means greater than at least one value, that is, greater than the minimum.

```
Select * From dbo.Customer
where Age>ANY ( Select age from dbo.Customer1)
```

	Id	FName	LName	CountryShortName	Age
1	3	Sukhi	Singh	NULL	25
2	4	James	Smith	CA	60
3	5	Robert	Ladson	US	54
4	6	Alice	John	US	87

We got the same records what were returned by our first query.
If you will use =ANY that is equal to IN. With ANY you can use different comparison operators such as = , <> , != , > , >= , !> , < , <= , !<

How to use ALL Logical Operator in SQL Server

ALL Logical operator returns TRUE if all of a set of comparisons are TRUE. ALL compares a scalar value with a single column set of values.

Let's understand ALL with examples.

Scenario:
Think about a scenario where we have two tables dbo.Customer and dbo.Customer1. Both tables has the column Age. If you need to get all the records from dbo.Customer table where Age is greater than maximum value of Age column in dbo.Customer1 table.What would be your query.

Solution:
We can use subquery and max function to write our query for above requirement. Let's create the tables first.

```
--Create Customer Table
Create table dbo.Customer
(Id int,
  FName VARCHAR(50),
  LName VARCHAR(50),
  CountryShortName CHAR(2),
  Age tinyint)
GO
--Insert Rows in dbo.Customer Table
insert into dbo.Customer
Values (
1,'Raza','M','PK',20),
(2,'Rita','John','US',12),
(3,'Sukhi','Singh',Null,25),
```

```
(4,'James','Smith','CA',60),
(5,'Robert','Ladson','US',54),
(6,'Alice','John','US',87)

--Create dbo.Customer1 table
Create table dbo.Customer1
(Id int,
  FName VARCHAR(50),
  LName VARCHAR(50),
  CountryShortName CHAR(2),
  Age tinyint)
GO
--Insert rows in dbo.Customer1 Table
insert into dbo.Customer1
Values
(7,'Raza','M','US',33),
(8,'Dita','M','US',15),
(9,'Adita','M','US',29)
```

1) Get all the records from dbo.Customer table where Age is greater than maximum Age value of dbo.Customer1 table by using Subquery and Max function.

	Id	FName	LName	CountryShortName	Age
1	4	James	Smith	CA	60
2	5	Robert	Ladson	US	54
3	6	Alice	John	US	87

2) Using ALL with SubQuery

For above requirement we can use ALL logical operator. In that case we don't have to use Max function. ALL is going compare our outer query value to set of

values from subquery. We can use >All, >ALL means greater than every value returned by subquery, In other words greater than max value.

```
Select * From dbo.Customer
where Age> All ( Select age from dbo.Customer1)
```

	Id	FName	LName	CountryShortName	Age
1	4	James	Smith	CA	60
2	5	Robert	Ladson	US	54
3	6	Alice	John	US	87

With ALL you can use different comparison operators such as = , <> , != , > , >= , !> , < , <= , !<

How to use EXISTS Logical Operator in SQL Server

Exists returns TRUE if a subquery contains any rows. EXISTS is used when we want to test for the existence of rows specified by a subquery.

Let's create dbo.Customer and dbo.Country Table and then use EXISTS to return records for different scenarios.

```
Create table dbo.Customer
(Id int,
  FName VARCHAR(50),
  LName VARCHAR(50),
  CountryShortName CHAR(2),
  Age tinyint)
GO
insert into dbo.Customer
Values (
1,'Raza','M','PK',20),
(2,'Rita','John','US',12),
(3,'Sukhi','Singh',Null,25),
(4,'James','Smith','CA',60),
(5,'Robert','Ladson','US',54),
(6,'Alice','John','US',87),
(7,'Raza','M','US',33),
(8,'Dita','M','US',15),
(9,'Adita','M','US',29)
```

```
Create table dbo.Country (
CId tinyint,
CountryName VARCHAR(50),
CountryShortName CHAR(2))
```

```
go
Insert into dbo.Country
Values
(1,'Pakistan','Pk'),
(2,'United States of America','US')
```

1) EXISTS will return TRUE if subquery contains any rows.

That is right, if our subquery will return any rows and we have used EXISTS, the outer query is going to return all rows.

```
Select * from dbo.Customer

WHERE Exists (Select 1)
```

	Id	FName	LName	CountryShortName	Age
1	1	Raza	M	PK	20
2	2	Rita	John	US	12
3	3	Sukhi	Singh	NULL	25
4	4	James	Smith	CA	60
5	5	Robert	Ladson	US	54
6	6	Alice	John	US	87
7	7	Raza	M	US	33
8	8	Dita	M	US	15
9	9	Adita	M	US	29

Noticed that our subquery is static query (Select 1). As subquery did return row and EXISTS returned TRUE so all the records from dbo.Customer table are displayed.

2) Use EXISTS and Join with SubQuery

The more real time example of EXISTS would be when we want to find all the

records from dbo.Customer table which has matching CountryShortName from dbo.Country Table.

```
SELECT *
FROM dbo.Customer a
WHERE EXISTS
   (SELECT 1
    FROM dbo.Country b
    WHERE a.CountryShortName=b.CountryShortName)
```

Noticed that I have compared CountryShortName from dbo.Customer and dbo.Country. Each outer row is going to be compared with subquery results and if matches, then we get the row.

We can use the IN clause for same requirement.

```
SELECT *
FROM dbo.Customer a
WHERE a.CountryShortName IN
   (SELECT b.CountryShortName
    FROM dbo.Country b
    WHERE a.CountryShortName=b.CountryShortName)
```

	Id	FName	LName	CountryShortName	Age
1	1	Raza	M	PK	20
2	2	Rita	John	US	12
3	5	Robert	Ladson	US	54
4	6	Alice	John	US	87
5	7	Raza	M	US	33
6	8	Dita	M	US	15
7	9	Adita	M	US	29

What is BETWEEN Logical Operator in SQL Server

BETWEEN returns TRUE if the operand is within range. BETWEEN logical operator is used when we want to return the row if operand is within range.

Scenario:
Let's say that we have dbo.Cutomer table and one of the column is Age. If we would like to return all the records from dbo.Customer table where age between 33 and 60. What Logical Operator we can use?

Solution:
We can use BETWEEN logical operator to test ranges. If True then row will be returned otherwise not.

```
Create table dbo.Customer
(Id int,
  FName VARCHAR(50),
  LName VARCHAR(50),
  CountryShortName CHAR(2),
  Age tinyint)
GO
insert into dbo.Customer
Values (
1,'Raza','M','PK',20),
(2,'Rita','John','US',12),
(3,'Sukhi','Singh',Null,25),
(4,'James','Smith','CA',60),
(5,'Robert','Ladson','US',54),
(6,'Alice','John','US',87),
(7,'Raza','M','US',33),
(8,'Dita','M','US',15),
(9,'Adita','M','US',29)
```

BETWEEN returns TRUE if the value of test_expression is greater than or equal to the value of begin_expression and less than or equal to the value of end_expression.

We can use below query to return all the rows if age is between 33 and 60.

```
Select * From dbo.Customer
where Age between 33 and 60
```

	Id	FName	LName	CountryShortName	Age
1	4	James	Smith	CA	60
2	5	Robert	Ladson	US	54
3	7	Raza	M	US	33

We can also use NOT BETWEEN that will retrun TRUE if the value of test expression is less than the value of begin expression or greater than the value of end expression.
If we want to return all the records where Age is less than 33 or greater than 60 we can use NOT BETWEEN as shown below.

```
Select * From dbo.Customer
where Age NOT between 33 and 60
```

	Id	FName	LName	CountryShortName	Age
1	1	Raza	M	PK	20
2	2	Rita	John	US	12
3	3	Sukhi	Singh	NULL	25
4	6	Alice	John	US	87
5	8	Dita	M	US	15
6	9	Adita	M	US	29

What is LIKE Logical Operator in SQL Server

LIKE logical operator is used when we want to return the row if operand matches a pattern. Like operator returns TRUE if the operand matches a pattern.

Sometime we need to perform pattern matching instead of equal or not equal. Like is used when we want to return the row if specific character string matches a specified pattern. Pattern can be combination of regular characters and wildcard characters.
To return the row back, regular characters must exactly match the characters specified in the character string. The wildcard characters can be matched with arbitrary parts of the character string.

Let's create dbo.Customer table and then create some real time examples

```
Create table dbo.Customer
(
  Id int,
  FName VARCHAR(50),
  LName VARCHAR(50),
  CountryShortName CHAR(2)
)
GO

insert into dbo.Customer
Values (
1,'Raza','M','PK'),
(2,'Rita','John','US'),
(3,'Sukhi','Singh',Null),
(4,'James','Smith','CA'),
```

(5,'Robert','Ladson','US'),
(6,'Alice','John','US'),
(7,'Raza','M','US'),
(8,'Dita','M','US'),
(9,'Adita','M','US')

1) Using %

Let's say if we want to find all the rows where FName contains "i" in them. We can use below query

```
Select * From dbo.Customer
where FName like '%i%'
```

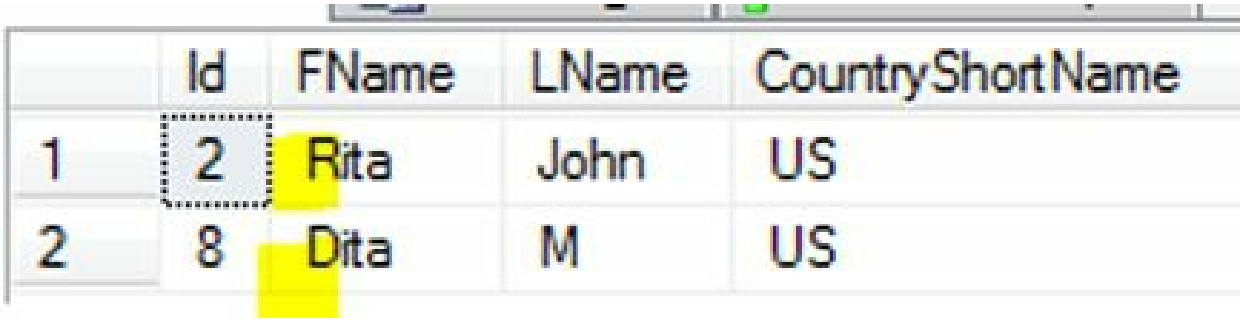
	Id	FName	LName	CountryShortName
1	2	Rita	John	US
2	3	Sukhi	Singh	NULL
3	6	Alice	John	US

Noticed that by using % before and after "i", we are telling the query to find all rows in which FName has "i" character and does not matter what other characters are before and after "i".

2) Using _ (underscore)

The underscore can be used when we want to check single character that can be anything and provide the rest of the characters for our match. Let's say that if I want to find all rows where FName first character can be anything but rest of them should be "ita". I can use below query.

```
Select * From dbo.Customer
where FName like '_ita'
```

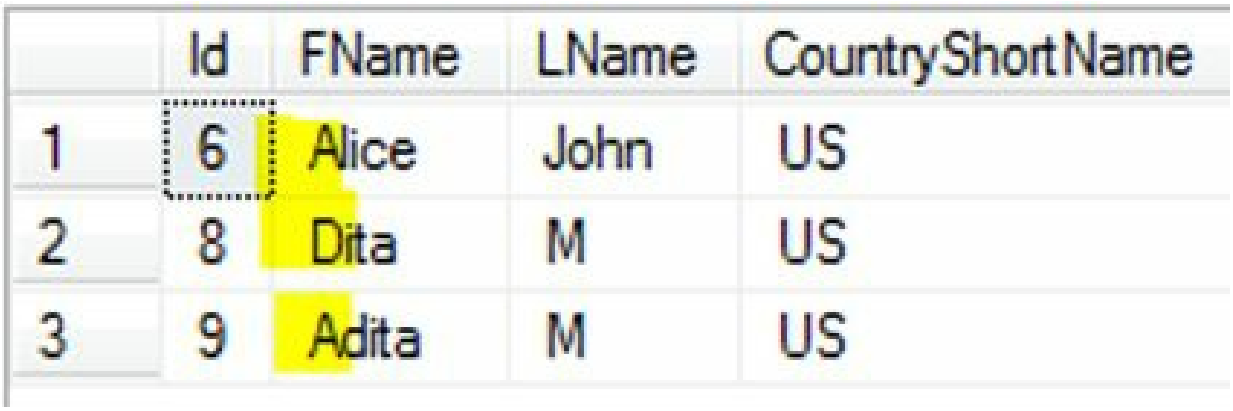


	Id	FName	LName	CountryShortName
1	2	Rita	John	US
2	8	Dita	M	US

3) Using [ ] - Any single character within the specified rang [a-t] or set [abc]

Like operator with [ ] can be used when we want to have range. Let's say if I want to find all the rows where FName first character start with [a-f]. We can use below query.

```
Select * From dbo.Customer
where FName like '[a-f]%'
```



	Id	FName	LName	CountryShortName
1	6	Alice	John	US
2	8	Dita	M	US
3	9	Adita	M	US

As you can see that I have used [a-f]%. That means I want the first character from a to f and after that any characters are fine as I used %.

4) [^] Any single character NOT within the specified rang [a-t] or set

[abc]

Let's say if I want to find all the rows where FName first character Dost NOT start with [a to f]. We can use below query.

```
Select * From dbo.Customer

where FName like '[^a-f]%'
```

	Id	FName	LName	CountryShortName
1	1	Raza	M	PK
2	2	Rita	John	US
3	3	Sukhi	Singh	NULL
4	4	James	Smith	CA
5	5	Robert	Ladson	US
6	7	Raza	M	US

Noticed that it only returned us the rows which does not start with any character from a-f.

Let's say that if we are want to get all the rows where FName does not start with a,d,j. we can use below query.

```
Select * From dbo.Customer

where FName like '[^adj]%'
```

	Id	FName	LName	CountryShortName
1	1	Raza	M	PK
2	2	Rita	John	US
3	3	Sukhi	Singh	NULL
4	5	Robert	Ladson	US
5	7	Raza	M	US

Filtering by OFFSET-FETCH Options in Select query

Scenario:
You are working as SQL Server developer with front end development team. The front end team needs to implement pagination. Confuse about Pagination? No problem. Think about viewing your bank statement or credit card statement. Where applications shows only 10 or 20 records per page and you have to click next to see next records. That is called pagination.

Now you understand pagination, the front end development needs SQL query from you that can be used to returned required results and they should be able to pass page number to return records.

Solution:
There are multiple ways to write Pagination queries, one of them is by using OFFSET FETCH clause. You have to sort the records if you want to use OFFSET FETCH.

Let's create dbo.TotalSale table and insert some sample records. I have only inserted 11 records.

```
CREATE TABLE [dbo].[TotalSale]
   (
    [id] [int] NOT NULL ,
    [SalePersonFName] [varchar](100) NULL ,
    [SalePersonLName] [varchar](100) NULL ,
    [ProductName] [varchar](100) NULL ,
    [ItemsSold] [int] NULL ,
    [SoldPrice] [float] NULL ,
    [SoldDate] [date] NULL ,
    [City] [varchar](100) NULL ,
    [State] [varchar](100) NULL ,
    [Country] [varchar](100) NULL ,
```

```
    [Region] [varchar](100) NULL
  )

INSERT  [dbo].[TotalSale]
       ( [id], [SalePersonFName], [SalePersonLName], [ProductName],
         [ItemsSold], [SoldPrice], [SoldDate], [City], [State], [Country],
         [Region] )
VALUES  ( 1, N'Aamir', N'Shahzad', N'TV', 1, 700, CAST(N'2015-07-15' AS
DATE),
         N'Charlotte', N'NC', N'USA', N'North America' )
 ,     ( 2, N'M', N'Raza', N'Cell Phone', 2, 800, CAST(N'2015-07-15' AS
DATE),
         N'Charlotte', N'NC', N'USA', N'North America' )
 ,     ( 3, N'Christy', N'Ladson', N'TV', 3, 1600,
         CAST(N'2015-04-02' AS DATE), N'High Point', N'NC', N'USA',
         N'North America' )
 ,     ( 4, N'John', N'Rivers', N'Laptop', 5, 2400,
         CAST(N'2014-03-09' AS DATE), N'Jersey City', N'NJ', N'USA',
         N'North America' )
 ,     ( 5, N'Najaf', N'Ali', N'Computer', 1, 300,
         CAST(N'2015-06-20' AS DATE), N'Karachi', N'Sindh', N'Pakistan',
         N'Asia' )
 ,     ( 6, N'Sukhjeet', N'Singh', N'TV', 2, 900, CAST(N'2015-06-21' AS
DATE),
         N'ChandiGar', N'Punjab', N'India', N'Asia' )
 ,     ( 7, N'Chirag', N'Patel', N'Cell Phone', 5, 1500,
         CAST(N'2015-06-23' AS DATE), N'AhmadAbad', N'Gujrat', N'India',
         N'Asia' )
 ,     ( 8, N'Aleena', N'Aman', N'Laptop', 2, 800,
         CAST(N'2015-05-25' AS DATE), N'Lahore', N'Punjab', N'Pakistan',
         N'Asia' )
 ,     ( 9, N'Petra', N'Henry', N'TV', 10, 5000, CAST(N'2015-04-08' AS
DATE),
         N'Paris', N'Île-de-France', N'France', N'Europe' )
 ,     ( 10, N'Rita', N'Roger', N'Laptop', 7, 2100,
         CAST(N'2015-04-11' AS DATE), N'Paris', N'Île-de-France', N'France',
```

```
        N'Europe' )
,     ( 11, N'Tamara', N'Tony', N'Cell Phone', 2, 1200,
        CAST(N'2015-03-03' AS DATE), N'Frankfurt', N'Hesse', N'Germany',
        N'Europe' )
```

1) Lets say if we would like to skip first 5 rows and want to show all rest of the rows we can use below query.

```
Select
[id], [SalePersonFName], [SalePersonLName], [ProductName],
[ItemsSold], [SoldPrice]
from dbo.TotalSale
        order by id
        OFFSET 5 rows
```

	id	SalePersonFName	SalePersonLName	ProductName	ItemsSold	SoldPrice
1	6	Sukhjeet	Singh	TV	2	900
2	7	Chirag	Patel	Cell Phone	5	1500
3	8	Aleena	Aman	Laptop	2	800
4	9	Petra	Henry	TV	10	5000
5	10	Rita	Roger	Laptop	7	2100
6	11	Tamara	Tony	Cell Phone	2	1200

2) Now if we would like to show 3 records per page, we can use below query. In this case we are going to show first page

```
Select
[id], [SalePersonFName], [SalePersonLName], [ProductName],
[ItemsSold], [SoldPrice]
from dbo.TotalSale
        order by id
        OFFSET 0 ROWS FETCH NEXT 3 ROWS ONLY;
```

	id	SalePersonFName	SalePersonLName	ProductName	ItemsSold	SoldPrice
1	1	Aamir	Shahzad	TV	1	700
2	2	M	Raza	Cell Phone	2	800
3	3	Christy	Ladson	TV	3	1600

Noticed that I have OFFSET 0, that means that I want to show first page and with 3 rows. If I want to show second page records, I will set to OFFSET 1, the Next 3 ROWS ONLY part will stay the same as want to show only 3 rows per page.

We can use variables so we don't have to make changes in the query and by changing the value of variables, we can return our required results. You can create Stored Procedure if you like by using below query.

```
Declare @PageNumber int
Declare @RowsPerPage int
set @RowsPerPage=3
SET @PageNumber=1

Select
[id], [SalePersonFName], [SalePersonLName], [ProductName],
[ItemsSold], [SoldPrice]
from dbo.TotalSale
        order by id
       OFFSET (@PageNumber-1)*@RowsPerPage ROWS FETCH NEXT
@RowsPerPage ROWS ONLY;
```

If we need to provide Stored Procedure to Front End team, which accepts page number and number of rows they would like to return for each page, you can use below to create Stored Procedure.

```
 Create procedure dbo.sp_GetSaleRecordsPerPage
```

```
@PageNumber int,
@RowsPerPage int
AS
BEGIN
Select
[id], [SalePersonFName], [SalePersonLName], [ProductName],
[ItemsSold], [SoldPrice]
from dbo.TotalSale
        order by id
       OFFSET (@PageNumber-1)*@RowsPerPage
       ROWS FETCH NEXT @RowsPerPage ROWS ONLY;
END
```

Let's say if we would like to return second page with 4 records, we can use the dbo.sp_GetSaleRecordsPerPage by providing below parameter values.

```
EXEC dbo.sp_GetSaleRecordsPerPage 2,4
```

	id	SalePersonFName	SalePersonLName	ProductName	ItemsSold	SoldPrice
1	5	Najaf	Ali	Computer	1	300
2	6	Sukhjeet	Singh	TV	2	900
3	7	Chirag	Patel	Cell Phone	5	1500
4	8	Aleena	Aman	Laptop	2	800

How to get random rows from SQL Server Table

Scenario:
You are working as SQL Server developer. You are asked to provide sample random data from dbo.Customer table. You might asked to provide random 100 rows or some percent of total data from table. What query you will use to provide required output?

Solution:
We can use the Top clause but as we are asked to provide the random records, we need to sort them randomly first. We can use newid() function in order by clause to sort them randomly.

Let's create dbo.Customer table with some sample data.

```
Create table dbo.Customer
(
  Id int,
  FName VARCHAR(50),
  LName VARCHAR(50),
  CountryShortName CHAR(2)
)
GO

Insert into dbo.Customer
Values (
1,'Raza','M','PK'),
(2,'Rita','John','US'),
(3,'Sukhi','Singh',Null),
(4,'James','Smith','CA'),
(5,'Robert','Ladson','US'),
```

(6,'Alice','John','US')

1) Use NewID ( ) in Order by to get random records

Let's say if we are interested to get 3 random records from dbo.Customer table, we can use below query.

```
Select top 3 * From dbo.Customer
order by NEWID()
```

	Id	FName	LName	CountryShortName
1	5	Robert	Ladson	US
2	4	James	Smith	CA
3	2	Rita	John	US

You can also use percent if you like as shown below

```
Select top 30 percent * From dbo.Customer
order by NEWID()
```

	Id	FName	LName	CountryShortName
1	1	Raza	M	PK
2	6	Alice	John	US

2) By using TABLESAMPLE SYSTEM

As per Microsoft Books Online "TABLESAMPLE SYSTEM returns an approximate percentage of rows and generates a random value for each physical 8-KB page in the table. Based on the random value for a page and the percentage specified in the query, a page is either included in the sample or excluded. Each page that is included returns all rows in the sample result set".

From here you can understand that if you have small table with few pages, you might not want to use TableSample as it will include or exclude the entire page. With few records in table, you might want to use method 1 and for large tables you can use TableSample.

If I would run below query on my dbo.Customer table, sometime I will get no records and when get the records, it will return all the records as they are placed on single page.

```
Select * From dbo.Customer tablesample (30 percent)
```

You can also use Rows you want with table sample as shown below. The rows returned can vary. You can limit them by using top n in select query.

```
Select * From dbo.Customer tablesample (2 rows)
```

When I executed above query on dbo.Customer table with total of 6 rows. It either returned me no rows or all six of them.
If you want to limit, you can use below query. Once again, I will suggest to use TableSample with big table where you have data on multiple data pages.

```
Select top 2 * From dbo.Customer tablesample (2 rows)
```

How to use Case Statement for Conditional Formatting in Select Query

Scenario:
You are working as SQL Server developer, you need to query dbo.Customer table that has CountryShortName column with country abbreviations. You want to generate another columns with Country Full name by using CountryShortName column values. How would you do that?

Solution:
You can use Case expressions to generate this new column depending upon the value of CountryShortName. In our example we are using only single columns but you can use multiple columns and check for multiple conditions.

Let's create dbo.Customer table with some sample data and then we will write our Select statement with Case expression.

```
Create table dbo.Customer
(Id int,
  FName VARCHAR(50),
  LName VARCHAR(50),
  CountryShortName CHAR(2))
GO

insert into dbo.Customer
Values (
1,'Raza','M','PK'),
(2,'Rita','John','US'),
(3,'Sukhi','Singh',Null)
```

1) You can use Column Name for which you want to check the values right after Case as shown below. Then write all the conditions on that column and finally use End as NewColumnName

```
Select
FName,
LName,
CountryShortName,
Case CountryShortName
When 'Pk' Then 'Pakistan'
When 'US' Then 'United States of America'
When 'IN' Then 'India'
Else 'Not Provided'
End AS CountryFullName
From dbo.Customer
```

	FName	LName	CountryShortName	CountryFullName
1	Raza	M	PK	Pakistan
2	Rita	John	US	United States of America
3	Sukhi	Singh	NULL	Not Provided

If you don't like to use Else part, you can remove that but in case when you will have value and it does not match with your conditions, It will return Null. In my case, If value does not match with my conditions, I would like to show as "Not Provided" by using Else part.

2) Don't use Column Name right after Case keyword

You can also write the case statement as shown below. In below example, we did not write Column Name right after Case. In this case we have to type column after every When. This way of writing is used when you want to check conditions for multiple columns or range of values.

```
Select
FName,
LName,
CountryShortName,
Case
```

```
When CountryShortName='Pk' Then 'Pakistan'
When CountryShortName='US' Then 'United States of America'
When CountryShortName='IN' Then 'India'
Else 'Not Provided'
End AS CountryFullName
From dbo.Customer
```

	FName	LName	CountryShortName	CountryFullName
1	Raza	M	PK	Pakistan
2	Rita	John	US	United States of America
3	Sukhi	Singh	NULL	Not Provided

Understand Column Alias in Select Query in SQL Server

Scenario:
You are working as SQL Server developer, you need to extract the data from dbo.Customer table and provide in an Excel Sheet. Your table has difference columns such as FName, LName but you want the output to come with FirstName and LastName. How would you do that?

Solution:
Alias is a great way to make your column names more readable. As in our question above, we want to display FName as FirstName and LName as LastName, Alias is the way to go. Let's create dbo.Customer table with columns and then use select query with alias to display Columns as we like.

```
Create table dbo.Customer
(
  Id int,
  FName VARCHAR(50),
  LName VARCHAR(50)
)
GO

insert into dbo.Customer
Values (
1,'Raza','M'),
(2,'Rita','John')
```

To create Alias you can use "AS" after column name or simply provide the alias right after column name. In below query, I have use "as" for FName but did not use for LName.

```
Select FName as FirstName,
LName LastName
From dbo.Customer
```



First Name	Last Name
Raza	M
Rita	John

If you would like to have space between your Alias Column Name such as "First Name". You can use [ ] around the alias or double quotes e.g "First Name".

```
Select FName as [First Name],
LName as "Last Name"
From dbo.Customer
```



First Name	Last Name
Raza	M
Rita	John

How to use Top with Ties in SQL Server

According to BOL

"WITH TIES Used when you want to return two or more rows that tie for last place in the limited results set. Must be used with the ORDER BY clause. WITH TIES may cause more rows to be returned than the value specified in expression. For example, if expression is set to 5 but 2 additional rows match the values of the ORDER BY columns in row 5, the result set will contain 7 rows."

Let's create dbo.Customer table and find out the difference between Top and Top With Ties.

```
Create table dbo.Customer
(
Id int,
Name VARCHAR(10),
Address VARCHAR(100)
)

--Insert records in dbo.Customer table
Insert into dbo.Customer
Values(1,'Aamir','XyZ Address')
,(2,'Raza',Null)
,(1,'Aamir','XyZ Address')
,(1,'John','XyZ Address')
,(5,'Lisa','XyZ Address')
```

Noticed that we have duplicate records as highlighted above. Let's run the below queries, first with only Top and second with "With Ties" and observe the output.

```
Select top (2) * From dbo.Customer
order by ID
```

	Id	Name	Address
1	1	Aamir	XyZ Address
2	1	Aamir	XyZ Address

```
Select top (2) WITH TIES  * From dbo.Customer
order by ID
```

	Id	Name	Address
1	1	Aamir	XyZ Address
2	1	Aamir	XyZ Address
3	1	John	XyZ Address

The first query returned only 2 records but second query with "With Ties" returned three records as the value for Id for third record was also 1 that matched with second row (id column value=1). If you use "With Ties" with Top, the query will also give you all ties with the last row based on the ordering column.

How to get Top / Bottom N Records from SQL Server Table in SQL Server

Scenario:
You are working as SQL Server or TSQL Developer, you need to get top/bottom x rows from a table. How would you do that?

Solution:
You can use TOP clause in your query to return top x records. Top clause is really helpful to take a quick look into some data points instead of returning all the records.

The syntax for top is

Select TOP Number | Percent column1, column2, column3... From SchemaName.TableName

Let's create our sample table dbo.Customer with sample data and try TOP.

```
--Create dbo.Customer table
Create table dbo.Customer(
Id int,
Name VARCHAR(10),
Address VARCHAR(100))

--Insert records in dbo.Customer table
Insert into dbo.Customer
Values(1,'Aamir','XyZ Address')
,(2,'Raza',Null)
,(3,'Rita','XyZ Address')
,(4,'John','XyZ Address')
,(5,'Lisa','XyZ Address')
```

1) Let's say if we would like to get Top 3 records from dbo.Customer with all columns, we can use below query.

```
Select top 3 * from dbo.Customer
```

Id	Name	Address
1	Aamir	XyZ Address
2	Raza	NULL
3	Rita	XyZ Address

The select query with top returned us top 3 records successfully. You can also sort the records and then get top x records by using Top.

2) Sort the records by Name and get top 3 records from dbo.Customer table.

```
Select top 3 * from dbo.Customer
Order by Name
```

Id	Name	Address
1	Aamir	XyZ Address
4	John	XyZ Address
5	Lisa	XyZ Address

3) How to get Bottom n Rows from table

As we have seen that we can sort the records first and then get top x rows. If we have a column such as id (auto incremental), sequence number or createdOn (datetime) column, we can sort by desc and then get the top x rows that will return us the bottom rows.
In our case we have Id, if we sort as desc and then get Top 3, we will be able to get bottom 3 records.

```
Select top 3 * from dbo.Customer
Order by ID desc
```



Id	Name	Address
5	Lisa	XyZ Address
4	John	XyZ Address
3	Rita	XyZ Address

4) Use Percent with Top

Let's say if we would like to get 50% of records from dbo.Customer table, we can use percent with Top in our select query. As we have only 5 records in our dbo.Customer table, 50% will be 2.5, 2.5 rows can't be returned so SQL will round to 3 and we will get 3 row.

```
Select top 50 Percent * from dbo.Customer
```



Id	Name	Address
1	Aamir	XyZ Address
2	Raza	NULL
3	Rita	XyZ Address

Again, Order by clause can be used with Percent the same way we did use with Top Number in example 1, 2 and 3.

How to restore Databases from backups files automatically in SQL Server

Scenario:
Let's say that you are working as SQL Server DBA and you tons of backups files sitting in one of the folder, you need to restore all those backups files to newly created DEV, QA or UAT server. If you will start restoring one after one manually, it will take you forever. You are looking for some scripts those can restore these full backups files to newly created server.

Solution:
You will be able to use this type of script to refresh databases on regular basis such as monthly or on demand from Production to lower environments.

I have tested this script on SQL Server 2016. You might get some error in case you are using old versions or latest versions as sometime Microsoft change the number of columns returned by RESTORE FILELISTONLY.

Check the columns list and update the definition of temp tables according to your version, rest of the script will be same.

I did write same type of script before but I was considering the databases will have only mdf and ldf. The below script can handle database with NDF as well and overwrite existing DB if exists.

```
--Provide the folder path where backups are present

DECLARE @BackupFolder VARCHAR(128)
SET @BackupFolder='E:\Backup\'

--Provide the data and log files paths
--where you would like to restore databases
```

```
Declare @DataFilesPath VARCHAR(128)
Declare @LogFilesPath VARCHAR(128)

SET @DataFilesPath='D:\Data\'
SET @LogFilesPath='D:\Logs\'

--Get Backup files informaiton in temp table
  IF OBJECT_ID('tempdb..#BackupFileList') IS NOT NULL
    DROP TABLE #BackupFileList

--Drop temp table if exits
IF OBJECT_ID('tempdb..#RESTOREFILELISTONLY') IS NOT NULL
    DROP TABLE #RESTOREFILELISTONLY

IF OBJECT_ID('tempdb..#RestoreHeaderOnly') IS NOT NULL
    DROP TABLE #RestoreHeaderOnly

  CREATE TABLE #BackupFileList (
    Id int identity(1,1),
    BackupFile nvarchar(255),
    Depth smallint,
    FileFlag bit)


    --Select * From #BackupFileList
--Store Backup information
Create Table #RestoreHeaderOnly(
BackupName nvarchar(128) ,
BackupDescription nvarchar(255) ,
BackupType smallint ,
ExpirationDate datetime ,
Compressed bit ,
Position smallint ,
DeviceType tinyint ,
UserName nvarchar(128) ,
ServerName nvarchar(128) ,
DatabaseName nvarchar(128) ,
```

```
DatabaseVersion int ,
DatabaseCreationDate datetime ,
BackupSize numeric(20,0) ,
FirstLSN numeric(25,0) ,
LastLSN numeric(25,0) ,
CheckpointLSN numeric(25,0) ,
DatabaseBackupLSN numeric(25,0) ,
BackupStartDate datetime ,
BackupFinishDate datetime ,
SortOrder smallint ,
CodePage smallint ,
UnicodeLocaleId int ,
UnicodeComparisonStyle int ,
CompatibilityLevel tinyint ,
SoftwareVendorId int ,
SoftwareVersionMajor int ,
SoftwareVersionMinor int ,
SoftwareVersionBuild int ,
MachineName nvarchar(128) ,
Flags int ,
BindingID uniqueidentifier ,
RecoveryForkID uniqueidentifier ,
Collation nvarchar(128) ,
FamilyGUID uniqueidentifier ,
HasBulkLoggedData bit ,
IsSnapshot bit ,
IsReadOnly bit ,
IsSingleUser bit ,
HasBackupChecksums bit ,
IsDamaged bit ,
BeginsLogChain bit ,
HasIncompleteMetaData bit ,
IsForceOffline bit ,
IsCopyOnly bit ,
FirstRecoveryForkID uniqueidentifier ,
ForkPointLSN numeric(25,0) NULL ,
```

```
RecoveryModel nvarchar(60) ,
DifferentialBaseLSN  numeric(25,0) NULL ,
DifferentialBaseGUID uniqueidentifier ,
BackupTypeDescription nvarchar(60) ,
BackupSetGUID uniqueidentifier NULL ,
CompressedBackupSize bigint ,--2008 has till here

containment tinyint not NULL )--2012 has till here

--KeyAlgorithm nvarchar(32),--Add below if using sql 2014,2016

--EncryptorThumbprint varbinary(20) ,

--EncryptorType nvarchar(32))
--Select * From #RESTOREFILELISTONLY

--Create Temp Table for DB files

--Select * From #RESTOREFILELISTONLY
--Create Temp Table for DB files
Create table #RESTOREFILELISTONLY(
LogicalName NVARCHAR(128),
PhysicalName NVARCHAR(260),
Type CHAR(1),
FileGroupName NVARCHAR(128),
Size numeric(20,0),
MaxSize numeric(20,0),
Field bigint,
CreateLSN numeric(25,0),
DropLSN numeric(25,0),
UniqueId uniqueidentifier,
ReadonlyLSN numeric(25,0),
ReadWriteLSN numeric(25,0),
BackupSizeInBytes BigInt,
SourceBlockSize Int,
FileGroupId int,
LogGroupGUID uniqueidentifier,
```

```
DifferentialBaseLSN numeric(25,0),
DifferentialBaseGUID uniqueidentifier,
IsReadOnly bit,
IsPresent bit,
TDEThumprint varbinary(32))  --2012 is till here
--SnapshotURL nvarchar(360))

--Save backup files into temp table
   INSERT INTO #BackupFileList (BackupFile,Depth,FileFlag)
   EXEC xp_dirtree @BackupFolder, 10, 1

   --Select * FROM #BackupFileList

   --Use Cursor to loop throught backups files and restore
Declare @BackupFile VARCHAR(500)

   DECLARE Cur CURSOR FOR
  SELECT BackupFile from #BackupFileList
  where fileflag=1

OPEN Cur
FETCH Next FROM Cur INTO @BackupFile
WHILE @@FETCH_STATUS = 0
  BEGIN

    Truncate table #RESTOREFILELISTONLY

--Insert data from RESTORE FILELISTONLY
insert into #RESTOREFILELISTONLY
EXEC('RESTORE FILELISTONLY FROM DISK =
'''+@BackupFolder+@BackupFile+''')

insert into #RestoreHeaderOnly
EXEC('RESTORE HEADERONLY FROM DISK =
'''+@BackupFolder+@BackupFile+''')
--Select * From #RESTOREFILELISTONLY
--Select * From #RestoreHeaderOnly
```

```
Declare @FilesNumber varchar(10)=Null
Declare @LogFileName NVARCHAR(128)=NULL
Declare @DataFileName NVARCHAr(128)=NULL
Declare @DBName NVARCHAR(128)=NULL
--We are considering we have a log and single data file
SET @LogFileName=(Select logicalName from #RESTOREFILELISTONLY
where type='L')
SET @DataFileName=(Select logicalName  from
#RESTOREFILELISTONLY where type='D' and
FileGroupName='PRIMARY')
SET @DBName=(Select top 1 DatabaseName from #RestoreHeaderOnly)
SET @FilesNumber=(Select count(*) From #RestoreHeaderOnly)


--Prepare Restore Statement
Declare @SQLRestore NVARCHAR(MAX)
SET @SQLRestore='RESTORE DATABASE ''' +@DBName+'''
  FROM DISK='''+@BackupFolder+@BackupFile+'''
WITH File='+@FilesNumber+',REPLACE,
MOVE '''+@DataFileName+''' TO
'''+@DataFilesPath+@DataFileName+'.mdf'',
MOVE '''+@LogFileName+''' TO '''+@LogFilesPath+@LogFileName+'.ldf'''

--Execute SQL to Restore DBs
Print @SQLRestore
EXEC(@SQLRestore)

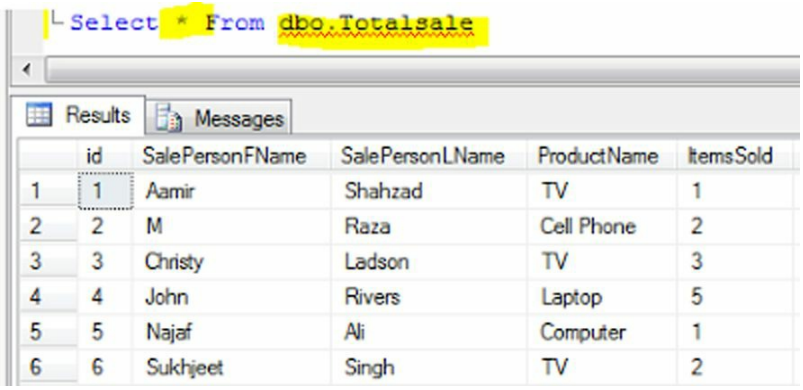
Truncate table #RestoreHeaderOnly
FETCH Next FROM Cur INTO @BackupFile
  END
CLOSE Cur
DEALLOCATE Cur
```

How to quickly write Select Query in SQL Server

1) Select all the columns from a table/ View by using *

You can always use Select * from SchemaName.TableName to display data for all the columns from a table or view. When you use * in Select query, it return you all the columns from table or view.

Let's say that if I want to select all the columns from dbo.TotalSale table, I can use below query



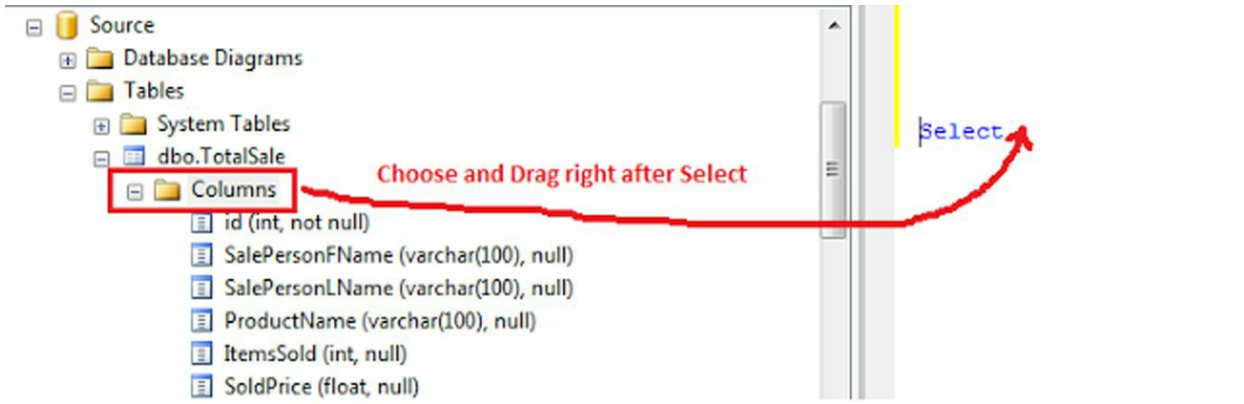
```
Select * From dbo.Totalsale
```

Results | Messages

	id	SalePersonFName	SalePersonLName	ProductName	ItemsSold
1	1	Aamir	Shahzad	TV	1
2	2	M	Raza	Cell Phone	2
3	3	Christy	Ladson	TV	3
4	4	John	Rivers	Laptop	5
5	5	Najaf	Ali	Computer	1
6	6	Sukhjeet	Singh	TV	2

2) Drag the Columns Tab to new query

If you would like to get all the column names for your select query, you will open new query window. Then type Select and Drag the Columns Tab under the table to query window. All the columns will be displayed. If you would like to remove few, you can remove.

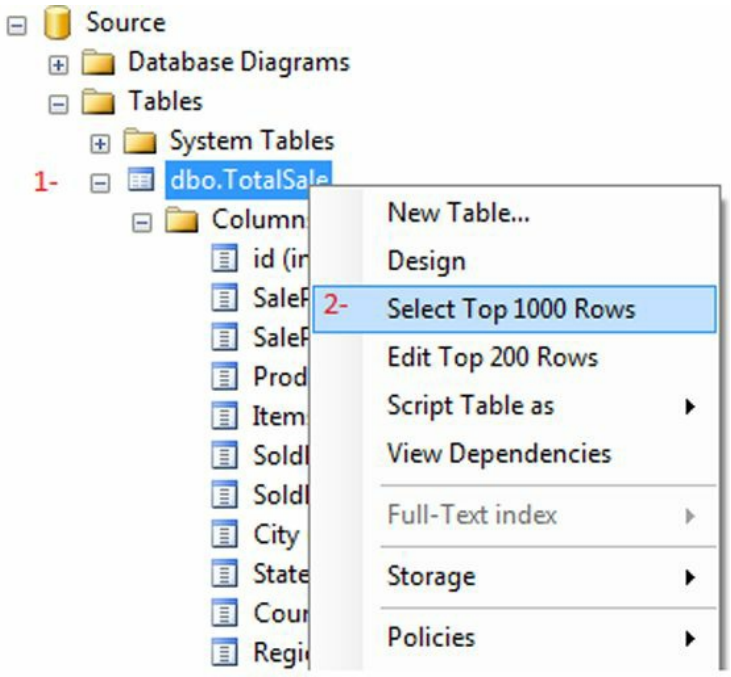


After dragging the Columns Tab, you will see the column names are added to select.



3) Select Top X Row

Third and easy way to write your select query is, Right click on Table name and then choose Select Top X Rows (X can be different as per your SSMS option settings). Once you choose it will generate the select query with all the column. You can remove the top x part from select and modify as required.

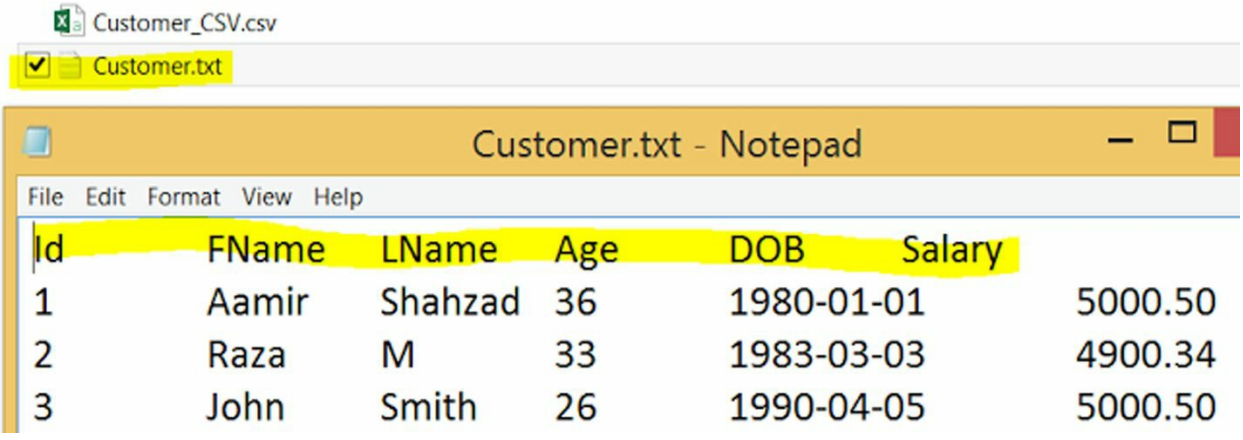

Source
Database Diagrams
Tables
System Tables
1-
dbo.TotalSale
Column
id (in
SaleP
SaleP
Prod
Item
Soldl
Soldl
City
State
Cour
Regi
New Table...
Design
2-
Select Top 1000 Rows
Edit Top 200 Rows
Script Table as
View Dependencies
Full-Text index
Storage
Policies

How to generate Insert Statements from Text Files for SQL Server Table in SQL Server

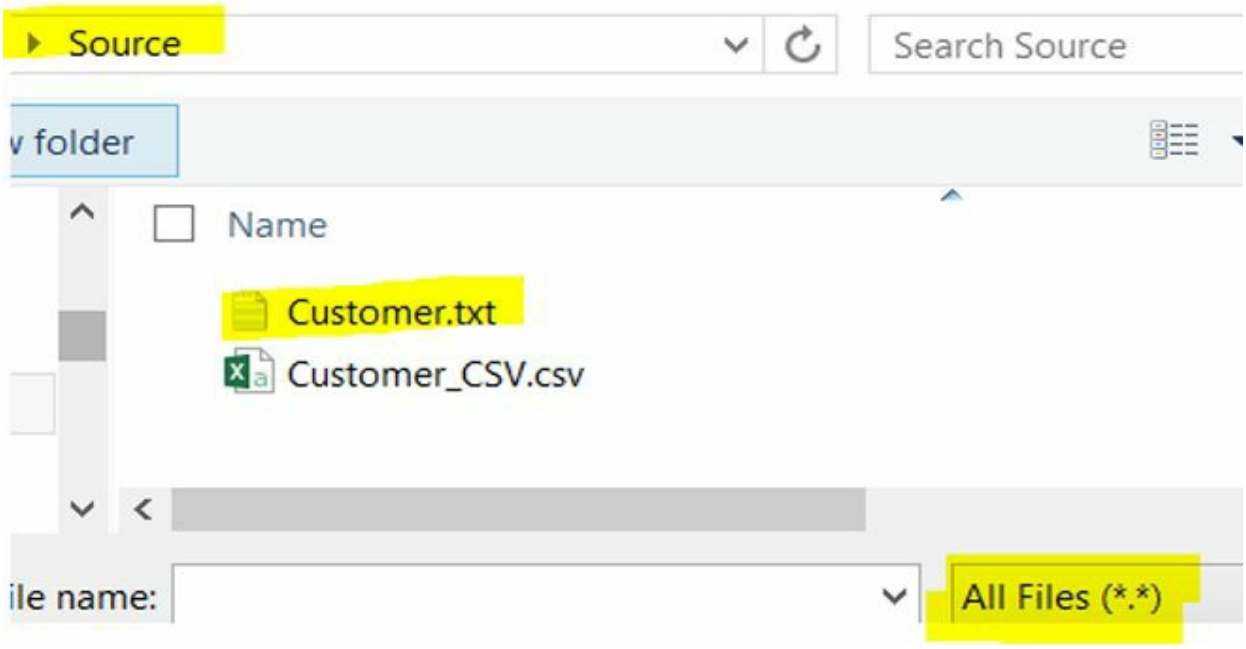
Scenario:
You are working as SQL Server developer, you get text files such as comma separate values, Tab delimited or pipe delimited files. It is one time task and you need to generate insert statements from these files so you can load the data into dbo.Customer table in DEV, QA, UAT and Production environments by using those scripts.

Solution:
If you have tab delimited or pipe delimited file, first of all we need to open with excel and then use the formulas.

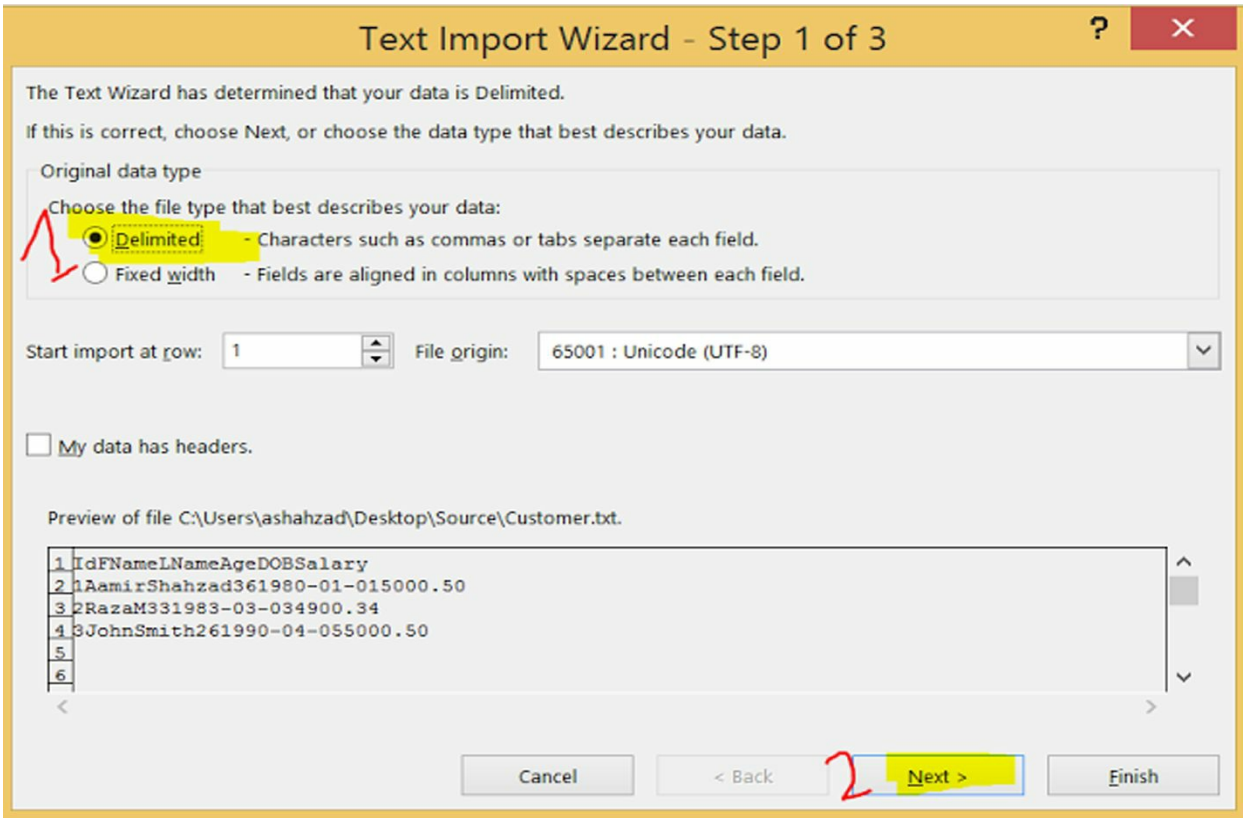
Step 1:
Let's say that I have Tab delimited Customer.txt file as shown below and want to open in excel.



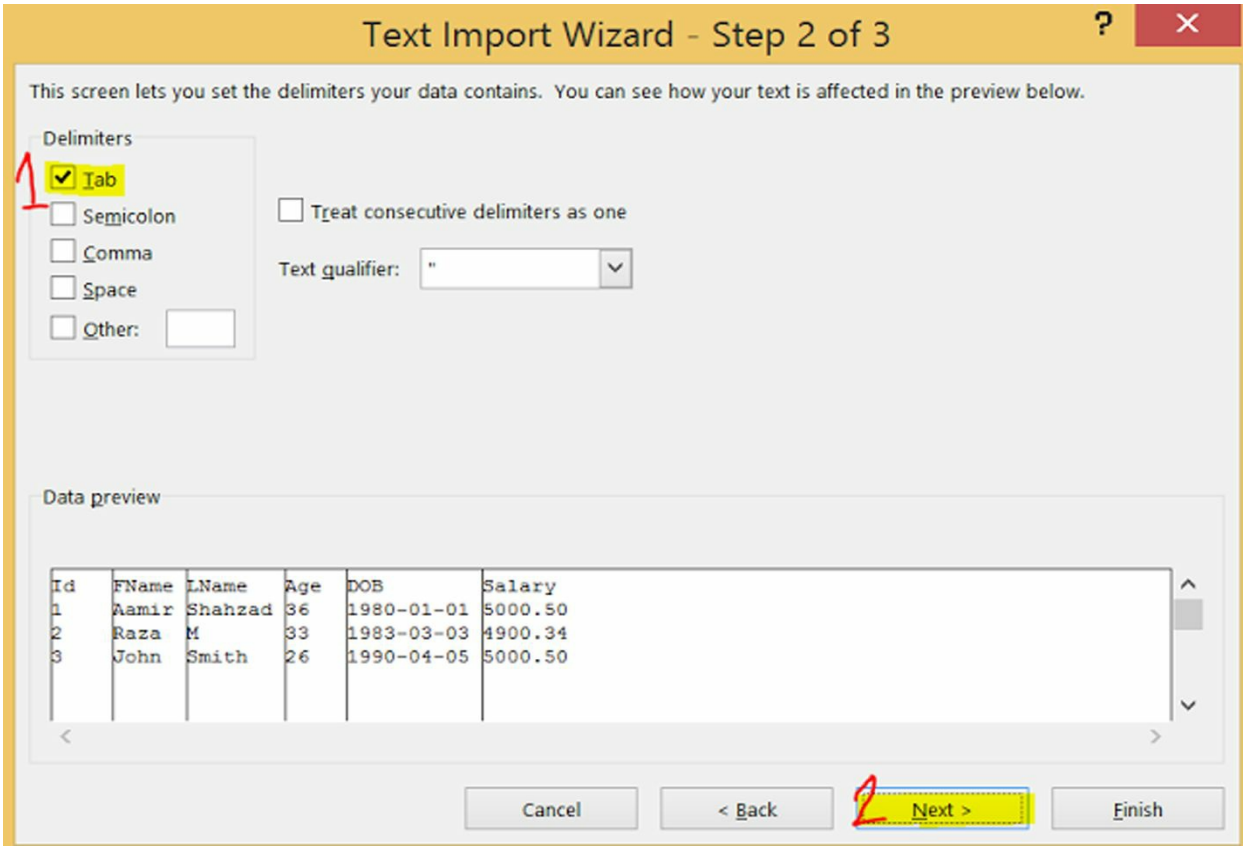
Step 2:
Open Excel and then go to File and hit Open, Browse to customer.txt file

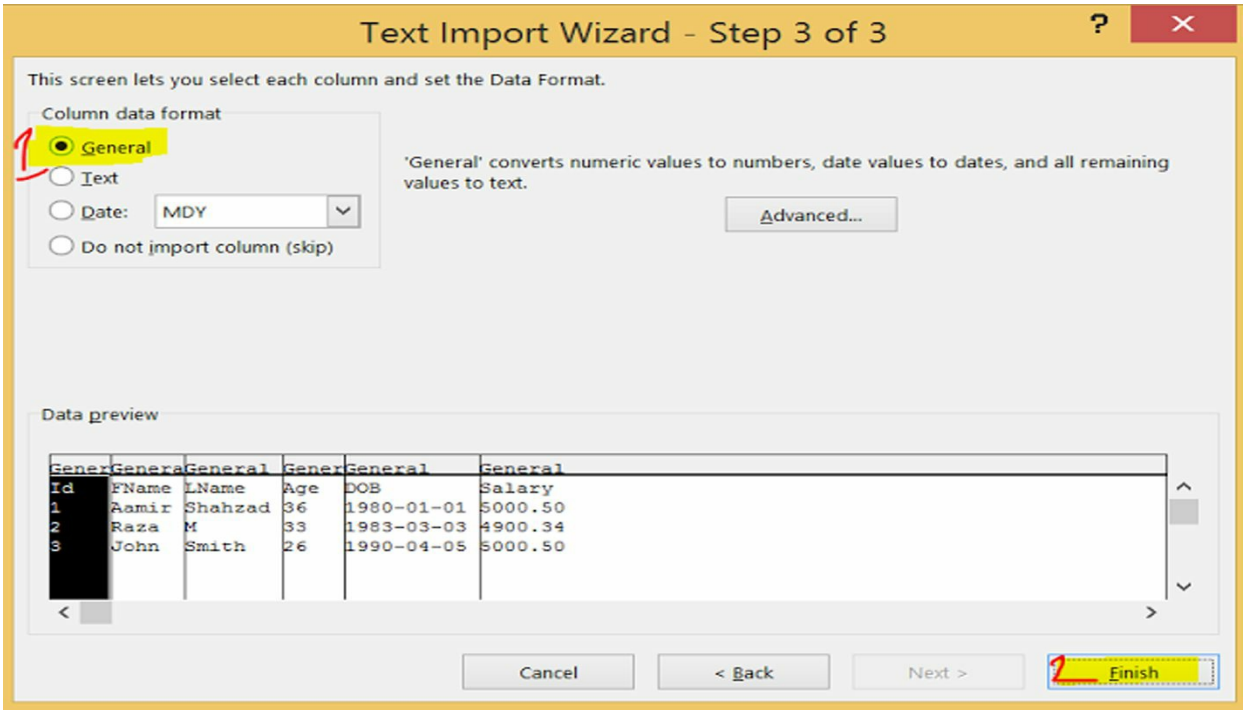


Test Import Wizard will open, Choose Delimited and Click My Data has Headers if it does and then Click Next



Choose the Delimiters, in our case it is Tab and hit Next.





Once you will hit Finish, below excel sheet will be populated with flat file data.

A	B	C	D	E	F
Id	FName	LName	Age	DOB	Salary
1	Aamir	Shahzad	36	1/1/1980	5000.5
2	Raza	M	33	3/3/1983	4900.34
3	John	Smith	26	4/5/1990	5000.5

More to Come

Write a short review at the site you purchased Email me at sales@meetcoogle.com with a link to your review and I'll reply with series next book for free!